

3 - Java

Introduzione

Java è un linguaggio di programmazione orientato agli oggetti e indipendente dalla piattaforma di esecuzione, garantita dal fatto che il codice non dipende dalla macchina fisica o dal sistema operativo specifico, ma viene compilato in un formato intermedio chiamato **bytecode**, il quale può essere poi eseguito su qualunque Virtual Machine compatibile. A differenza del paradigma procedurale, che descrive i sistemi come un insieme di processi tramite flow-chart e utilizza procedure e funzioni tipiche di linguaggi come C, Basic o Pascal, il mondo a oggetti descrive i sistemi come un insieme di "cose" modellate attraverso gerarchie e dipendenze tra classi. Questo moderno approccio utilizza costrutti basati su dichiarazioni di classi e metodi.

Il paradigma Object-Oriented

Riprendendo la definizione del paradigma orientato agli oggetti, ogni singola entità del mondo reale è considerata un oggetto caratterizzato da uno stato e da un funzionamento specifico.

☰ Esempio di paradigma OO nel mondo reale

Ad esempio, lo stato di una bicicletta è definito da valori come la marcia corrente e la velocità di marcia, mentre il suo funzionamento comprende azioni concrete come il cambio della marcia o la frenata

Nel contesto del software, gli oggetti reali vengono modellati memorizzando il loro stato all'interno di **fields** (ovvero variabili o attributi) ed esponendo il loro funzionamento all'esterno attraverso **methods** (funzioni o metodi), che hanno il compito cruciale di modificare lo stato dell'oggetto agendo in modo mirato sui suoi attributi.

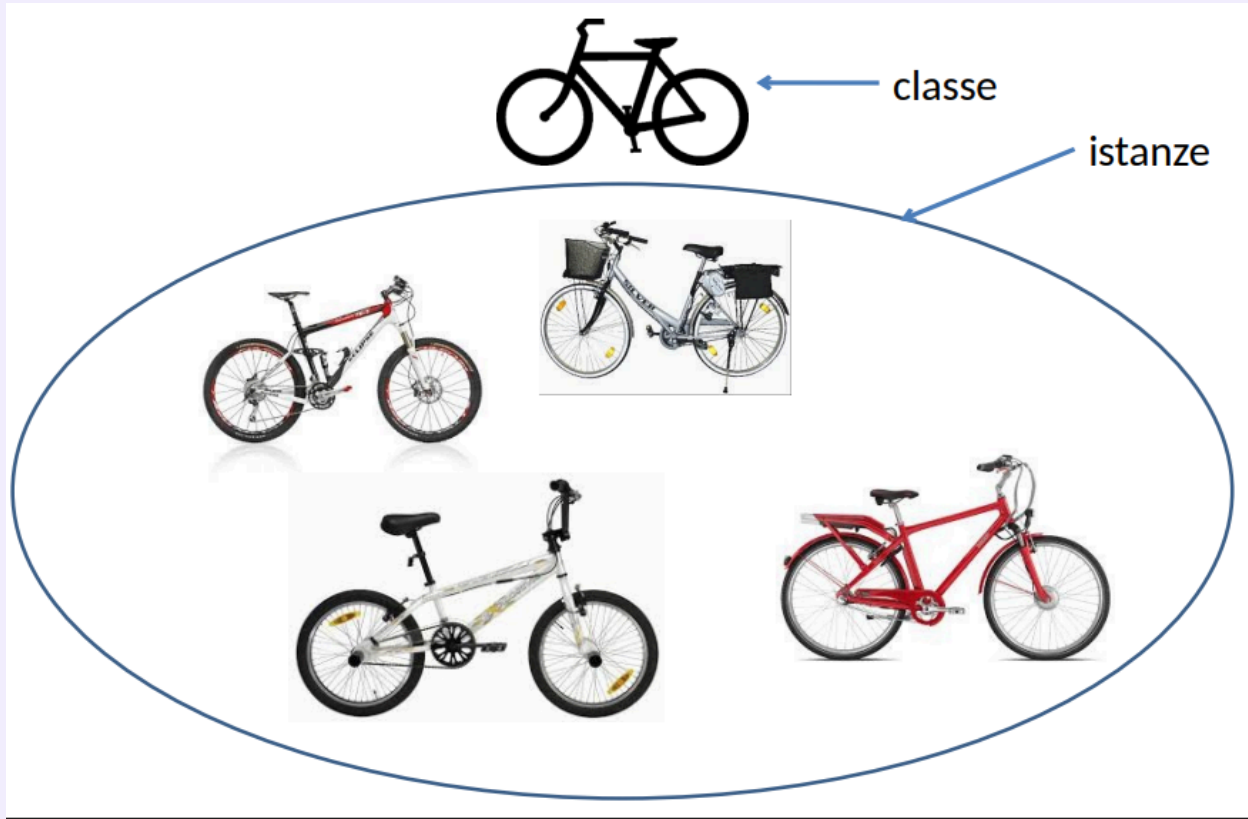
L'adozione di questo paradigma porta numerosi e importanti vantaggi:

- Favorisce la modularità, permettendo di gestire il codice di ogni oggetto in modo separato e indipendente;
- Garantisce l'information-hiding, nascondendo i dettagli implementativi e permettendo l'interazione solo tramite metodi prestabiliti;
- Facilita il riutilizzo del codice, consentendo di estendere o sostituire facilmente oggetti scritti da altri sviluppatori.

Classi, Ereditarietà e Package

Poiché nel mondo reale moltissimi oggetti condividono delle caratteristiche intrinseche, nella programmazione essi vengono raggruppati in **classi** che ne definiscono il funzionamento e le peculiarità comuni, rendendo quindi i singoli oggetti istanze fisiche e operative di tali classi.

☰ Esempio di classe

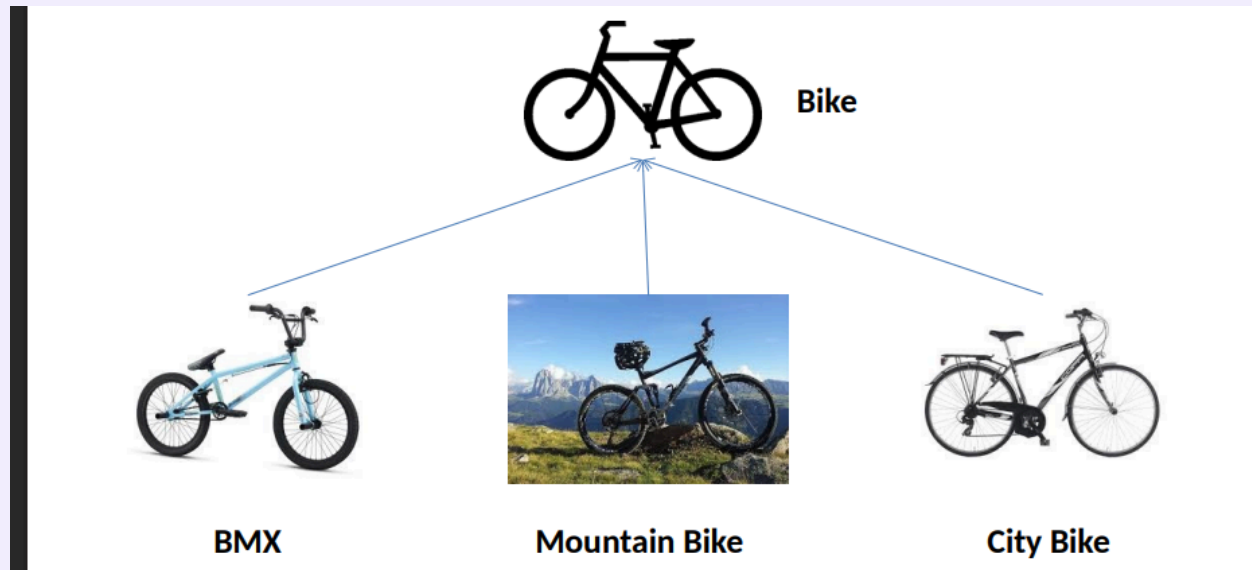


L'ereditarietà è un meccanismo estremamente potente che permette di rappresentare le gerarchie tra classi, consentendo a una determinata classe di ereditare in blocco gli attributi e i metodi di un'altra classe genitore.

☰ Esempio di ereditarietà

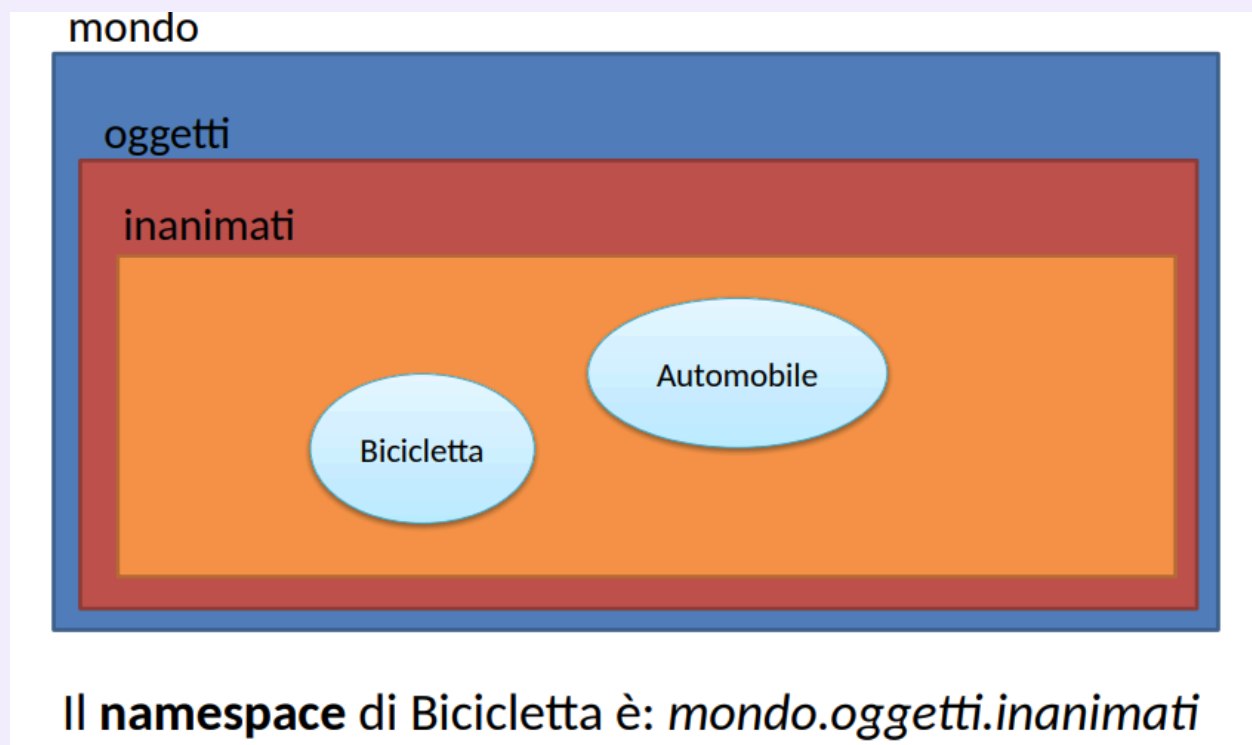
Riprendendo l'esempio di prima, classi più specifiche come le BMX, le Mountain Bike e le City Bike ereditano tutte le caratteristiche di base di una Bicicletta generica,

aggiungendo poi i propri attributi e metodi specializzati.



Per gestire l'organizzazione di progetti complessi, Java permette di strutturare le classi all'interno di specifiche cartelle separate chiamate **package**, le quali supportano la creazione di ulteriori sottocartelle per definire gerarchie annidate. Il percorso completo e sequenziale dei package necessari per individuare univocamente una classe prende il nome formale di **namespace**

☰ Rappresentazione grafica di package e namespace sulle biciclette



Codice

Commenti

Per commentare dentro un codice Java, utilizziamo questi simboli:

```
/*
Questo è un commento multi riga, infatti posso anche andare a capo senza
farmi problemi.
Ecco qui!
*/

//Questo è un commento per una singola riga
```

Il compilatore Java ignorerà qualunque cosa posto all'interno degli slash per le righe con la prima sintassi, nella seconda soltanto la riga corrente

Definizione di classi, il main e println()

Per definire una classe in Java usiamo la sintassi **class <nomeclasse>**

☰ Esempio di definizione classe

```
class HelloWorldApp{
    public static void main(String[] args){
        System.out.println("Ciao mondo!")
    }
}
```

Il main è l'entrypoint di ogni applicazione dove possiamo richiamare altre classi e metodi, in Java deve essere sempre presente all'interno di un applicazione. Come argomento ha sempre `String[] args`, che si riferisce agli argomenti che possiamo passare al main dalla riga di comando quando invochiamo un applicazione Java.

La riga `System.out.println("Ciao mondo!")` è composta da 3 elementi fondamentali:

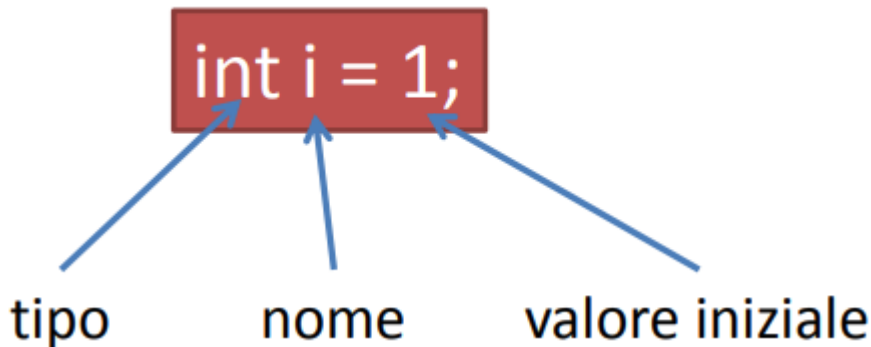
- `System`, che è una delle librerie **core** di Java
- `out` è un suo attributo (in questo caso un oggetto `PrintStream`)
- `println` è un metodo della classe `PrintStream` che permette di scrivere su un dispositivo di output

Le variabili

Le variabili in Java ricoprono il ruolo fondamentale di **descrivere lo stato di un oggetto**, ed esse possiedono sempre:

- Un tipo;
- Un nome identificativo;
- Un valore coerente con il tipo specificato in fase di dichiarazione.

Vanno sempre dichiarate, utilizzando la seguente sintassi:



Nel linguaggio esistono diverse categorie di variabili:

- Le variabili delle istanze (non statiche), in cui ogni singolo oggetto conserva il suo stato in modo indipendente;
- Le variabili delle classi (statiche), che risultano condivise e identiche per tutte le istanze di una specifica classe;
- Le variabili locali, che vengono utilizzate e risultano accessibili esclusivamente all'interno del metodo in cui sono dichiarate;
- I parametri, che rappresentano i valori passati al momento della chiamata di un metodo

Per quanto riguarda i nomi identificativi, il linguaggio impone che siano case-sensitive, che inizino rigorosamente con una lettera o con il simbolo \$ e che possano contenere numeri al loro interno, avendo preferibilmente delle nomenclature autoesplicative.

Le convenzioni comunemente accettate prevedono che

- Le costanti e le variabili statiche debbano essere scritte interamente a lettere maiuscole,
- Per i nomi composti da più parole si adotta lo stile in cui la prima lettera di ogni nuova parola successiva alla prima viene scritta in maiuscolo.

Tipi di dato primitivi

Il linguaggio Java definisce a livello nativo una serie di tipi di dato primitivi per ottimizzare la memorizzazione dei valori.

Tabella dei valori primitivi

Tipo	Descrizione
byte	8-bit, intero [-128, 127], valore di default <i>0</i>
short	16-bit, intero [-32.768, 32.767], valore di default <i>0</i>
int	32-bit, intero [-2^{31} , $2^{31}-1$], valore di default <i>0</i>
long	64-bit, intero [-2^{63} , $2^{63}-1$], valore di default <i>0</i>
float	32-bit IEEE 754 floating point, valore di default <i>0</i>
double	64-bit IEEE 754 floating point, valore di default <i>0</i>
boolean	true/false, valore di default <i>false</i>
char	16-bit Unicode character, \u0000-\uFFFF, valore di default \u0000

Quasi tutti i tipi primitivi possiedono uno zero numerico, un carattere nullo o il valore false come impostazione di default, ad eccezione delle variabili locali che risultano prive di tale assegnazione automatica e necessitano pertanto di un'inizializzazione manuale.

Ogni tipo primitivo è inoltre affiancato dalla sua rispettiva classe "wrapper" (la sua rappresentazione ad oggetti come Integer, Long o Boolean).

Tipo String

Il tipo String non si tratta di un tipo primitivo ma di un oggetto vero e proprio il cui valore di default risulta essere null. Le stringhe sono istanze immutabili e il linguaggio provvede a creare automaticamente un nuovo oggetto String ogni volta che incontra una sequenza di caratteri racchiusa tra doppi apici.

I letterali

Il termine letterale (**literal**) viene utilizzato per indicare i **valori espliciti che assegniamo direttamente nel codice alle variabili di tipo primitivo o alle stringhe**.

☰ Esempi di literals

```
boolean r = true;  
char c = 'Z';  
byte b = 100;  
short s = 10000;  
int i = 100000;
```

Letterali interi

Per quanto riguarda i letterali interi, il compilatore li interpreta sempre come tipo `int` di default, a meno che non venga specificato esplicitamente l'uso del formato long tramite l'aggiunta di una lettera L finale; essi supportano inoltre la scrittura in formati alternativi, potendo essere espressi anche in base esadecimale o binaria.

≡ Esempio di letterali interi

```
int n=10;  
long m=1000L;  
int h=0xa1;  
int b=0b001101;
```

Letterali in virgola mobile (o floating point)

I letterali in virgola mobile seguono una logica simile: sono considerati di tipo `double` per impostazione predefinita, a meno che non si utilizzi la lettera `f` per forzarne il tipo a `float`, e supportano l'impiego della notazione scientifica.

≡ Esempio di letterali float

```
float f=3.14f;  
double d1=134.54  
double d2=1.3454e2 //Notazione scientifica
```

Letterali caratteri e Stringhe (character e String)

i character possono contenere qualunque carattere Unicode a 16-bit racchiuso tra singoli apici, mentre le stringhe impiegano i doppi apici.

☰ Esempio di letterali caratteri e stringhe

```
Per char vale questo -> 'c'
Per String vale questo -> "testo di esempio"
```

All'interno dei testi è inoltre possibile avvalersi di speciali sequenze di escape per rappresentare agevolmente caratteri di controllo come il line feed, la tabulazione oppure per visualizzare simboli specifici come il doppio apice, il singolo apice e il backslash

📄 Lista di sequenze escape

```
\n, \r, \f, \b: line feed, carriage return, form feed, backspace
\t: tab
\\', \', \\: doppio apice, singolo apice, backslash
```

Array

L'array è un **oggetto** che contiene un numero finito di oggetti (o tipi primitivi) dello stesso tipo.

Ha una lunghezza fissa definita al momento della creazione, ogni suo elemento viene chiamato **elemento** ed è possibile accederne con l'indice

☰ Esempio di vari tipi di array

```
Dichiarazione di un array: float[] v;
Inizializzazione: float[] v=new float[100];
Assegnazione: v[3]=1.3;
Dichiarazione e inizializzazione contemporanea: int[] a = {10, 34,
21};
Array multidimensionali: double[][] m;
```

Quando si dichiara un array inizializzato, questo avrà dimensioni uguali a quanti elementi son stati inizializzati

Copia di un array

La classe `System` mette a disposizione il metodo `arraycopy` per copiare degli array

Metodo per la copia di un array

```
arraycopy(Object src, int srcPos, Object dest, int destPos, int  
length)
```

È composto da:

- **src**: array sorgente;
- **srcPos**: posizione di inizio in src;
- **dest**: array di destinazione;
- **destPos**: posizione di inizio in dest;
- **length**: numero di elementi da copiare;

Esempio di copia di array

```
int[] a = {10, 30, 20, 15, 45};  
int[] b = new int[3];  
System.arraycopy(a, 2, b, 0, 3);
```

```
b[0] = 20  
b[1] = 15  
b[2] = 45
```

Manipolazione di array

La classe `Arrays` mette a disposizione un serie di metodi statici per manipolare gli array, la documentazione è presente [qui](#), ma riportiamo comunque alcuni metodi come:

- **copyOfRange**: è la copia di array;
- **fill**: riempimento di array;

- **equals**: confronto tra due array;
- **search**: ricerca di un elemento;
- **sort**: ordinamento crescente;

Operatori

Gli operatori eseguono delle operazioni da 1 a 3 argomenti e restituiscono un valore. Gli operatori con stessa priorità son eseguiti da sinistra verso destra, **tranne per le operazioni di assegnamento**

Tabella operatori

Operatori	Precedenza
postfix	expr++, expr--
unary	++expr, --expr, +expr, -expr, ~, !
multiplicative	*, /, %
additive	+, -
shift	<<, >>, >>>
relational	<, >, <=, >=, instanceof

Operatori	Precedenza
equality	==, !=
bitwise AND	&
bitwise XOR	^
bitwise OR	
logical AND	&&
logical OR	
ternary	?, :
assignment	=, +=, -=, *=, /=, %=, &=, ^=, =, <<=, >>=, >>>=

Alcune considerazioni da fare sono:

- **unary**: permette di negare un numero binario;
- **shift**: shifta i bit in un numero
- **relational**: mettono in relazione due
- **equality**: i doppi uguale son particolari, con i tipi primitivi funzionano nella classica maniera, mentre nel confronto di oggetti funzionano solo se questi ultimi hanno lo stesso OID (Object ID)

Operatori aritmetici

Tabella degli operatori aritmetici

+	somma	int s = 3 + 4;
-	differenza	int d = 3 - 2;
*	moltiplicazione	int m = 3 * 2;
/	divisione	int d = 10 / 2;
%	modulo	int r = 11 % 5;

È possibile combinare assegnazione ed operatori aritmetici, per esempio:

```
x += 1; è equivalente a x = x + 1;
x *= 2; è equivalente a x = x * 2;
```

L'operatore `+` può essere utilizzato per concatenare le stringhe:

```
String a="Hello "; String b=" world!"; String msg = a + b; ("Hello
world!")
```

Operatori unari

Tabella degli operatori unari

+	indica i numeri positivi, può essere omesso	int a=+1;
-	indica i numeri negativi	int b=-a;
++	incremento	a++;
--	decremento	b--;
!	complemento logico, inverte il valore di un boolean	boolean b = true; boolean a = !b;

Operatori uguaglianza e relazionali

Operatore	Nome
==	uguale a
!=	diverso da
>	maggiore di
>=	maggiore o uguale a
<	minore di
<=	minore o uguale a

Questi operatori restituiscono un boolean come risultato del confronto tra due espressioni

Operatori condizionali

Operatore	Nome
&&	AND-logico
	OR-logico

In Java le espressioni sono valutate solo se necessario, per esempio se ci sono OR valuterà soltanto una e, se questa è vera, le altre verranno scartate a prescindere (in altri linguaggi questo non succede)

Operatore 'instanceof'

instanceof è un particolare operatore per stabilire se un oggetto è istanza di una particolare classe, come risultato restituisce un booleano.

Esempio di instanceof

```
String s = "sono una stringa";
boolean t = s instanceof String;
boolean f = s instanceof Double
```

Bitwise e bitshift

Gli operatori **bitwise** e di **bit shift** permettono di **manipolare direttamente i singoli bit a livello macchina**.

L'operatore tilde (~) calcola il complemento della rappresentazione binaria andando a trasformare tutti gli zeri in uno e viceversa.

Per quanto riguarda lo scorrimento dei bit (shift) troviamo:

- L'operatore << sposta i bit verso sinistra
- L'operatore >> li sposta verso destra
- L'operatore >>> si occupa di spostare i bit a destra ma senza conservare il segno originale

Java mette a disposizione specifici operatori binari sui bit, che sono l'AND logico, lo XOR e l'OR che troviamo all'inizio del capitolo .

Espressioni

Un'espressione in Java è un costrutto fondamentale composto tipicamente da **variabili**, **operatori** e **chiamate a metodi**

La caratteristica essenziale di un'espressione è che, una volta valutata, restituisce sempre un singolo risultato.

Il tipo di questo risultato non è universale, ma dipende strettamente dai tipi delle variabili coinvolte, dagli operatori utilizzati e, naturalmente, dai valori restituiti dagli eventuali metodi chiamati.

Per gestire priorità complesse, le parentesi possono essere utilizzate liberamente per definire e forzare un preciso ordine di valutazione degli operatori.

Blocchi

I blocchi sono gruppi di istruzioni racchiusi tra parentesi graffe

☰ Esempio di blocco di codice

```
if (condition) {
    // Inizio primo blocco
    System.out.println("La condizione è vera.");
}
```

```

}
// Fine primo blocco
else {
// Inizio secondo blocco
System.out.println("La condizione è falsa.");
}
// Fine secondo blocco

```

Statement

Uno **statement** rappresenta una **singola istruzione esecutiva** e la sua sintassi richiede tassativamente che termini sempre con il carattere punto e virgola (;).

All'interno del linguaggio si possono individuare quattro tipologie principali di statement:

1. Le istruzioni di assegnazione di valori;
2. Gli operatori matematici particolari come ++ o --;
3. Le istruzioni che effettuano una chiamata ad un metodo;
4. Le istruzioni dedicate alla creazione di un nuovo oggetto in memoria;

Controllo del flusso

if-then e if-then-else

Le istruzioni if-then e if-then-else sono essenziali per eseguire dei blocchi di codice solo ed esclusivamente se si verifica una determinata condizione.

☰ Esempio di istruzione if-then-else

```

if (condition) {
//blocco if
}
else {
//blocco else
}

```

L'if-then avviene soltanto quando una condizione è vera, altrimenti nel caso sia falsa questa va nell'else

if-else annidati

Quando la logica del programma lo richiede, è assolutamente possibile annidare più istruzioni if-else per creare percorsi decisionali multipli

```
if (condizione1) {  
    //blocco condizione 1  
}  
else if (condizione2){  
    //blocco condizione 2  
}  
else if (condizione3){  
    //blocco condizione 3  
}
```

Switch

Per gestire scenari con molteplici opzioni si utilizza l'istruzione **switch**, che definisce diversi percorsi di esecuzione in base allo specifico valore che assume una singola variabile

≡ Esempio di switch

```
switch (<variabile>) {  
    case <valore1>: ...; break; // istruzione 1  
    case <valore2>: ...; break; // istruzione 2  
    default: ...; break; // istruzione default  
}
```

Tale variabile viene confrontata con vari blocchi "case", prevedendo anche un caso "default" finale nel caso nessuna condizione sia rispettata.

Lo switch può essere applicato soltanto a tipi di dati ben precisi, ovvero ai tipi primitivi byte, short, int, char, alla classe String e ai tipi enumerativi

While

Il ciclo while permette di eseguire un blocco di istruzioni finché una specifica condizione posta in testa al ciclo si mantiene vera

≡ Esempio di condizione while

```
while (condizione) {  
    //blocco istruzioni while  
}
```

}

Do while

Il ciclo do-while è concettualmente simile al while, ma con una profonda differenza, ossia il controllo della condizione viene effettuato solamente alla fine del blocco di codice, facendolo sempre eseguire **almeno** una volta.

≡ Esempio do-while

```
do {
    //blocco istruzioni while
} while (condizione);
```

for

L'istruzione **for** permette di iterare un blocco di istruzioni su uno specifico intervallo di valori

≡ Esempio for

```
for (initialization; termination; increment) {
    statement(s) //blocco istruzioni
}
```

L'intestazione di questo ciclo racchiude tre parametri chiave separati da punto e virgola: la fase di inizializzazione, la condizione di terminazione e l'incremento (o decremento)

for, Array, Collection (for each)

Il for è spesso utilizzato per iterare sugli oggetti di un Array o di una Collection, in questo caso esiste una forma compatta (**for-each**)

≡ Esempio for-each

```
public static void main(String[] args){
    int[] numbers ={1,2,3,4,5,6,7,8,9,10};
    for (int item : numbers) {
        System.out.println("Count is: " + item);
    }
}
```



```

    }
}

```

A ogni passaggio del ciclo, la variabile temporanea `item` assume automaticamente il valore dell'elemento successivo, dal primo all'ultimo, fermandosi da sola quando l'array è terminato

Il `for-each` **non garantisce nessun ordinamento**, serve soltanto a visitare quello presente all'interno del contenitore

break

Lo statement `break` interrompe istantaneamente e definitivamente un ciclo

≡ Esempio di statement break

```

for (i = 0; i < arrayOfInts.length; i++) { //si cerca
    if (arrayOfInts[i] == searchfor) {
        foundIt = true;
        break;
    }
}

```

Nel caso di cicli annidati, è importante sapere che il `break` va a interrompere solamente il ciclo più innestato rispetto alla posizione dell'istruzione stessa,

continue

Lo statement `continue` salta la corrente iterazione per passare a quella successiva

≡ Esempio di statement continue, cercando in una stringa solo le 'p'

```

for (int i = 0; i < max; i++) {
    // interested only in p's
    if (searchMe.charAt(i) != 'p')
        continue;
    // process p's
    numPs++;
}

```

Spesso viene attivato da un costrutto condizionale, come in questo caso

return

L'istruzione return viene adoperata per terminare in modo definitivo l'esecuzione di un metodo corrente.

A seconda del metodo, può essere usata in modo autonomo e senza restituire alcun valore, oppure restituendo un valore specifico tramite una variabile

Classi

Dichiarazione di una classe

La dichiarazione di una classe in Java serve a definire la struttura fondamentale che comprende lo stato della classe attraverso gli attributi, le modalità di creazione e inizializzazione tramite i costruttori, e le funzionalità offerte mediante i metodi

Definizione di una classe in Java

```
class MyClass {  
    // attributi  
    // costruttori  
    // metodi  
}
```

A livello sintattico, la dichiarazione di una classe inizia specificando la sua visibilità, seguita dalla parola chiave class e dal nome della classe stessa

Definizione di una classe completa

```
<visibilità> class <nome> extends <superclass>  
implements <interface1>, <interface2> ... {  
    //body  
}
```

Durante questa fase è anche possibile indicare da quale superclasse essa erediti, utilizzando la parola chiave **extends**, e definire se implementa una o più interfacce tramite la parola chiave **implements** (le vedremo successivamente).

Il livello di visibilità, che può essere public, private o protected, determina come e da chi la classe può essere vista e utilizzata all'interno del progetto

Variabili all'interno

All'interno di una classe possiamo trovare diverse tipologie di variabili:

- I **field**, o **attributi**, sono variabili che definiscono lo stato dell'oggetto e risultano visibili in tutta la classe
- Le **variabili locali**, sono confinate e visibili esclusivamente all'interno del metodo in cui vengono dichiarate e utilizzate
- I **parametri** sono variabili speciali che vengono passate a un metodo nel momento in cui questo viene richiamato

Dichiarazione degli attributi

Quando si dichiara un attributo, è necessario specificarne la visibilità, il tipo di dato e il nome; se non viene assegnato esplicitamente un valore iniziale, la variabile assumerà il valore di default previsto per il suo tipo

≡ Esempio di variabili all'interno di una classe

```
public class Bicycle {  
    private int gear = 1;  
    private int speed;  
}
```

Metodi

Dichiarazione

I metodi rappresentano le funzioni di una classe e la loro dichiarazione richiede l'indicazione della visibilità, del tipo di dato restituito, del nome e di eventuali parametri racchiusi tra parentesi

≡ Esempio di dichiarazione di metodi

```
public int sum(int a, int b) { //tipo, nome e parametri  
    return a+b;  
}
```

Nome

Per convenzione, i nomi dei metodi dovrebbero essere scritti in minuscolo e iniziare preferibilmente con un verbo, poiché indicano un'azione; se il nome è composto da più parole, si utilizza la notazione camelCase, inserendo la lettera maiuscola per identificare l'inizio di ogni parola successiva alla prima

Overloading di metodi

Java supporta l'overloading dei metodi, una funzionalità che permette a una classe di avere più metodi con lo stesso nome, a condizione fondamentale che abbiano una lista di parametri differente per numero o tipo.

È importante notare che i metodi sovraccaricati con lo stesso nome devono comunque restituire lo stesso tipo di valore.

☰ Esempio di overloading

```
public int sum(int a, int b) {  
    return a+b;  
}  
  
public int sum(int a, int b, int c) {  
    return a+b+c;  
}
```

Parametri

Per quanto riguarda i parametri, ogni metodo può riceverne da zero a molti, e questi possono essere di qualsiasi tipo, spaziando dai tipi primitivi fino agli oggetti di altre classi come array o stringhe

I parametri all'interno della firma del metodo devono avere nomi differenti tra loro e non è consentito dichiarare variabili locali all'interno del metodo che abbiano lo stesso nome di un parametro. È invece possibile che un parametro abbia lo stesso nome di un attributo della classe, tuttavia in questo scenario l'attributo viene "nascosto" e reso non direttamente visibile al metodo, a meno che non si utilizzi in modo esplicito la parola chiave **this** per disambiguare il riferimento.

Costruttori

I costruttori sono metodi speciali progettati appositamente per inizializzare gli oggetti di una classe al momento della loro creazione.

☰ Esempio di un costruttore

```
// Metodo senza valori
public Bicycle(int startSpeed, int startGear) {
    gear = startGear;
    speed = startSpeed;
}
// Metodo con valori
public Bicycle() {
    gear = 1;
    speed = 0;
}
```

Essi si distinguono per avere lo stesso identico nome della classe a cui appartengono e per il fatto di non restituire alcun valore, nemmeno void.

Esattamente come accade per i metodi, una classe può dichiarare più di un costruttore sfruttando l'overloading, purché non esistano due costruttori con lo stesso numero e tipo di parametri. Anche ai costruttori possono essere applicati i modificatori di visibilità **private**, **public** o **protected**.

Istruzione 'this'

L'istruzione **this** è uno strumento fondamentale che permette di fare riferimento in modo esplicito agli attributi e ai costruttori della classe corrente in cui ci si trova, utile per risolvere ambiguità di nomi o per richiamare un costruttore dall'interno di un altro costruttore

☰ Esempio di istruzione 'this'

```
public class Bicycle {
    private int gear;
    private int speed;
    public Bicycle(int startSpeed, int startGear) {
        this.gear = startGear;
        this.speed = startSpeed;
    }

    public Bicycle() {
        this(3, 0);
    }
}
```

Oggetti

Gli oggetti veri e propri prendono vita in memoria utilizzando l'istruzione **new**, la quale si occupa di invocare il costruttore appropriato della classe a cui l'oggetto deve appartenere. Una volta che l'oggetto è stato istanziato, è possibile accedere ai suoi attributi e invocare i suoi metodi sfruttando la sintassi basata sul punto.

≡ Esempio di istanza e utilizzo di un oggetto

```
Bicycle myBike = new Bicycle(0, 3); //Istanza di un nuovo oggetto

myBike.gear; //richiamo dell'attributo
myBike.getSpeed(); //Richiamo del metodo
```

Numero arbitrario di argomenti

Un'ulteriore caratteristica avanzata permette di definire in un metodo un numero arbitrario di argomenti dello stesso tipo. In questo caso, all'interno del metodo, il parametro viene trattato a tutti gli effetti come un array, rappresentato dai 3 puntini

≡ Esempio di numero arbitrario di argomenti

```
public void print(String... s) { // s è visto come un array
    for (String item:s) {
        System.out.println(item);
    }
}

print("Pippo", "Topolino", "Pluto") // Posso invocare print passando
    tanti oggetti di tipo String
```

Metodi static

Il modificatore **static** viene impiegato per definire attributi e metodi a livello di classe anziché a livello di singola istanza. Questo significa che una variabile dichiarata come static è condivisa e mantiene lo stesso valore per tutte le istanze create da quella specifica classe. Di conseguenza, i metodi contrassegnati come static possono essere chiamati direttamente utilizzando il nome della classe, senza la necessità preventiva di creare un'istanza o un oggetto.

≡ Esempio con dichiarazione ed uso metodi static

```
public class Person {
    static int numbOfPersons=0; // Qui il numero di persone aumenterà
    // senza essere resettato alla prossima esecuzione
    private String name;
    private String surname;
    public Person(String name, String surname) {
        this.name=name;
        this.surname=surname;
        ++numbOfPersons;
    }
}

Person p1=new Person(''Pippo'', ''Rossi'');
Person p2=new Person(''Topolino'', ''Bianchi'');
System.out.println(Person.numbOfPersons); // Accedo al valore della
// variabile static della classe Person
```

Costanti

Per definire delle **costanti**, ovvero attributi il cui valore una volta assegnato non può più essere modificato, si utilizza il modificatore **final**. Un uso comune in Java per le costanti globali prevede la combinazione dei modificatori **public**, **static** e **final**, rendendo il valore accessibile ovunque senza dover istanziare la classe

```
private final int A = 3; //Visibile solo nella classe
public final int X = 4 // Visibile ad altre classi
public static final double PI=3.14 // Visibile ad altre classi in modo
// static (senza dover creare un'istanza della classe)
```

Classi innestate

Il linguaggio offre la possibilità di definire una classe direttamente all'interno del corpo di un'altra classe, creando le cosiddette **classi innestate**.

Queste si dividono principalmente in due categorie:

- **Classi innestate statiche:** Una classe innestata statica è indipendente e non ha accesso alle risorse e ai membri della classe esterna

- **Classi interne:** al contrario, una classe interna gode di un accesso privilegiato a tutte le risorse della classe esterna che la contiene, comprese quelle dichiarate con visibilità private

L'utilizzo delle classi innestate è consigliato principalmente in tre scenari:

- Quando la classe interna risulta utile esclusivamente alle logiche della classe esterna;
- Quando si desidera incapsulare una classe B dentro una classe A per permetterle di accedere alle risorse private di A mantenendole sicure;
- In generale per raggruppare logicamente il codice rendendolo più pulito e leggibile.

☰ Esempio di classi innestate

```
class OuterClass {
    ...
    static class StaticNestedClass { // Non ha accesso alle risorse di
OuterClass
        ...
    }
    class InnerClass { // Ha accesso alle risorse di OuterClass anche
se dichiarate private
        ...
    }
}
```

Tipi enumerativi

I tipi enumerativi permettono di definire dei tipi che possono assumere solo un set predefinito di costanti

☰ Esempio di tipo enumerativo

```
public enum Day {
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY,
    THURSDAY, FRIDAY, SATURDAY
}
```

Quando si crea un tipo enum, Java crea automaticamente una classe di tipo enum che mette a disposizione dei metodi e permette di definire anche dei costruttori e nuovi metodi.

Interfacce

Le interfacce in Java permettono di definire il funzionamento di una classe indicando esplicitamente quali metodi la classe deve possedere per potervi aderire.

Esse contengono unicamente la descrizione delle firme dei metodi senza fornirne alcuna implementazione, ma possono comunque includere delle costanti.

Dichiarazione di interfacce

```
public interface <nome> extends <interface1>, <interface2>,
<interface3> {
    // dichiarazioni di costanti
    // firme dei metodi
    void methodA(int i, double x);
    int methodB(String s);
}
```

Grazie alle interfacce è possibile definire gruppi di classi che condividono le stesse funzionalità, lasciando però a ciascuna di esse la libertà di implementarle in maniera differente

L'implementazione pratica da parte di una classe avviene tramite la parola chiave `implements`, la quale obbliga la classe stessa a fornire il codice per tutti i metodi definiti all'interno dell'interfaccia.

Nelle interfacce va rispettato le stesse regole di nome delle classi

Ereditarietà

In Java, le classi possono ereditare da altre classi, il che implica che la sottoclasse acquisisce automaticamente tutti gli attributi e i metodi che possiedono una visibilità `public` oppure `protected` all'interno della superclasse

Esempio di ereditarietà in una calcolatrice

Prendiamo una calcolatrice basica, come questa:

```
public class Calculator {  
    private double memory;  
    public Calculator() {  
    }  
    public Calculator(double memory) {  
        this.memory = memory;  
    }  
    public double getMemory() {  
        return memory;  
    }  
    public void setMemory(double memory) {  
        this.memory = memory;  
    }  
    public void resetMemory() {  
        this.setMemory(0);  
    }  
}
```

```
public double sum(double a, double b) {  
    return a + b;  
}  
public double diff(double a, double b) {  
    return a - b;  
}  
public double mul(double a, double b) {  
    return a * b;  
}  
public double div(double a, double b) {  
    return a/b;  
}  
}
```

Decidiamo di estenderla con una calcolatrice scientifica

```
public class ScientificCalculator extends Calculator {
    public double tan(double x) {
        return Math.tan(x);
    }
    public double sin(double x) {
        return Math.sin(x);
    }
}
```

ScientificCalculator estende Calculator. Quindi sarà di tipo Calculator ed erediterà tutti gli attributi e i metodi pubblici di Calculator

```
ScientificCalculator sc=new ScientificCalculator();
System.out.println(sc.sin(4));
System.out.println(sc.sum(3, 4));
```

ScientificCalculator eredita il metodo pubblico *sum* della classe padre Calculator

Un aspetto fondamentale del linguaggio è che tutte le classi scritte in Java ereditano, in modo diretto o indiretto, dalla classe madre universale chiamata `Object`

Overriding e poliformismo

Una classe ha la capacità di ridefinire un metodo che ha precedentemente ereditato dalla sua superclasse (o classe padre).

Quando questo accade, il metodo originale della superclasse viene nascosto e, al suo posto, viene invocato esclusivamente quello ridefinito all'interno della sottoclasse; questo specifico fenomeno strutturale prende il nome di **overriding**

Il polimorfismo è strettamente legato ai concetti di ereditarietà e overriding, in quanto rappresenta la **capacità di ogni sottoclasse di poter ridefinire specifici comportamenti della propria superclasse**.

Tutte le sottoclassi possono riscrivere determinati comportamenti per adattarli alle proprie esigenze, pur mantenendo altre caratteristiche operative in comune con la classe **padre**

≡ Esempio di overriding, poliformismo e ereditarietà

```
public class ClassA {
    public void printMe() {
        System.out.println("Io sono A");
    }
    public void sayHello() {
        System.out.println("Hello!");
    }
}

public class ClassB extends ClassA {
    public void printMe() { //override del metodo in A, ereditato
        System.out.println("Io sono B");
    }
}
```

Se qui andassi a richiamare il metodo specificando un oggetto di tipo B e metodo printMe, otterrei come output "Io sono B", poichè ho fatto overriding del metodo precedente

Accesso alla superclasse da una sottoclasse

Per interagire esplicitamente con gli elementi della superclasse, Java mette a disposizione dello sviluppatore la parola chiave `super`, questa quando viene utilizzata permette:

- Di invocare direttamente i costruttori della classe padre, utilizzando `super()` senza parametri o `super(lista parametri)` con argomenti specifici, \
- Di richiamare i metodi originali della superclasse tramite la sintassi `super.methodSuper(...)`

≡ Esempio di accesso alla superclasse da una sottoclasse

```
public class ClassA {
    public void printMe() {
        System.out.println("Io sono A");
    }
    public void sayHello() {
        System.out.println("Hello!");
    }
}
```

```

}

public class ClassB extends ClassA {
    public void printMe() {
        super.printMe() //qui invochiamo il metodo di A direttamente
        System.out.println("Io sono B");
    }
}

```

Classi astratte

Le classi astratte si distinguono per la presenza di metodi astratti, ovvero funzioni di cui viene fornita solamente la firma senza alcuna implementazione, sebbene la classe possa contenere dei metodi implementati

La caratteristica restrittiva principale di una classe astratta, dichiarata tramite la parola chiave `abstract`, è che non è fisicamente possibile istanziarne degli oggetti diretti, infatti esse sono concepite esclusivamente per essere ereditate; in tal caso, la sottoclasse ha l'obbligo di fornire un'implementazione concreta per tutti i metodi astratti che ha ereditato

≡ Esempio di dichiarazione classe astratta

```

public abstract class GraphicObject {
    // declare fields
    // declare nonabstract methods
    abstract void draw();
}

```

Classi astratte vs interfacce

Esistono differenze importanti tra classi astratte e interfacce:

1. Nelle classi astratte è possibile dichiarare attributi che non siano obbligatoriamente `static` e `final`, e si possono definire metodi completi della loro implementazione. All'interno delle interfacce tutti gli attributi devono essere rigorosamente `static` e `final`, e tutti i metodi assumono di default una visibilità `public`
2. In Java è consentito estendere una sola classe alla volta (anche se astratta), ma si possono implementare contemporaneamente numerose interfacce
3. Le classi astratte sono utili quando si desidera condividere del codice (sotto forma di metodi) tra un insieme di classi che sono tra di loro strettamente correlate, quando ci si

aspetta che le classi figlie abbiano molti metodi o attributi in comune, o quando si necessita di attributi non statici e non finali per permettere ai metodi di modificare liberamente lo stato degli oggetti. Le interfacce sono la scelta preferibile se le classi che le devono implementare non sono strettamente correlate, quando si vuole semplicemente specificare il comportamento di una particolare struttura dati senza entrare nei dettagli implementativi, o quando si ha il bisogno specifico di ricorrere e simulare un'ereditarietà multipli.

Classi astratte e Interfacce combinate

In Java, una classe astratta può implementare una o più interfacce. Il grande vantaggio in questo caso è che la classe astratta non è obbligata a implementare subito tutti i metodi richiesti, si può decidere di scriverne solo alcuni e lasciarne altri non implementati, le sottoclassi concrete (cioè le classi "normali" che andranno a estenderla) avranno poi l'obbligo di fornire il codice per tutti i metodi rimasti vuoti.

Numeri

Java mette a disposizione delle classi che rappresentano i tipi primitivi numerici che sono `Byte`, `Short`, `Long`, `Integer`, `Float`, `Double`. Ognuno di questi eredita dalla classe madre `Numbers`.

Il compilatore converte automaticamente tra tipi primitivi e classi usando una tecnica chiamata **boxing/unboxing**:

- Il **boxing** (noto anche come **wrapper**) è la trasformazione che consiste nel posizionare un tipo primitivo all'interno di un oggetto in modo che il valore possa essere utilizzato come riferimento.
- **Unboxing** è la trasformazione inversa dell'estrazione del valore primitivo dal suo oggetto nel wrapper

Questa classe mette a disposizione come sottoclassi:

- `BigInteger/BigDecimal`: sottoclassi adibite a interi e decimali con alta precisione
- `AtomicInteger/AtomicDecimal`: sottoclassi adibite a interi e decimali per applicazioni concorrenti, una volta che un'operazione viene effettuata su un tipo di questo genere non è interrompibile (da questo la sua atomicità)

Di solito si usa questa classe per alcuni motivi, tra cui:

- Un argomento deve essere un oggetto e non un tipo primitivo
- Quando bisogna definire delle costanti statiche nelle classi (come `MAX_VALUE` o `MIN_VALUE`)
- Per delle conversioni tra tipo, come
 - Da `String` (stringa contenente dei numeri) a tipo primitivo

- Da un tipo numerico ad un altro

La classe number implementa diversi metodi tra cui:

Metodi per la conversione in un tipo primitivo

- `byte byteValue()`
- `short shortValue()`
- `int intValue()`
- `long longValue()`
- `float floatValue()`
- `double doubleValue()`

Metodi per il confronto di un numero con un argomento

- `int compareTo(Byte anotherByte)`
- `int compareTo(Double anotherDouble)`
- `int compareTo(Float anotherFloat)`
- `int compareTo(Integer anotherInteger)`
- `int compareTo(Long anotherLong)`
- `int compareTo(Short anotherShort)`

`boolean equals(Object obj)`

Questo metodo serve per confrontare due oggetti booleani per determinare se siano di stesso valore. Restituisce "true" se sono uguali, altrimenti "false"

Metodi per la conversione

- `static Integer decode(String s)`: converte una stringa in `Integer`
- `static int parseInt(String s)`: converte una stringa in `int`
- `static int parseInt(String s, int radix)`: converte una stringa in `int`, (radix=sistema di numerazione 10, 2, 8, 16)
- `String toString()`: restituisce la stringa che rappresenta questo numero
- `static String toString(int i)`: restituisce la stringa che rappresenta il numero `i`
- `static Integer valueOf(int i)`: restituisce l'`Integer` che rappresenta il valore primitivo `i`
- `static Integer valueOf(String s)`: simile a `parseInt` ma restituisce un `Integer`
- `static Integer valueOf(String s, int radix)`: simile a `parseInt`

Stampa dei numeri

In Java, poiché ogni numero può essere convertito in una stringa, è possibile stamparli direttamente sullo standard output utilizzando i metodi `System.out.print` e `System.out.println`. `System.out` è un'istanza della classe `PrintStream`, il che permette di utilizzare indifferentemente anche i metodi equivalenti `printf` e `format`

☰ Esempio di format

```
System.out.format("The value of the float variable is %f, while the
value of the integer variable is %d, and the string is %s", floatVar,
intVar, stringVar)
```

Il metodo `format` accetta una stringa di formattazione seguita da una serie di argomenti, specificando in modo preciso come questi ultimi debbano essere visualizzati

📘 Flag converter e flag

Al metodo `format` possono essere passate queste flag:

Converter	Flag	Explanation
d		A decimal integer
f		A float
n		A new line character appropriate to the platform running the application. You should always use <code>%n</code> , rather than <code>\n</code>
tB		A date & time conversion—locale-specific full name of month
td, te		A date & time conversion—2-digit day of month. td has leading zeroes as needed, te does not
ty, tY		A date & time conversion—ty = 2-digit year, tY = 4-digit year
tl		A date & time conversion—hour in 12-hour clock.
tM		A date & time conversion—minutes in 2 digits, with leading zeroes as necessary.
tp		A date & time conversion—locale-specific am/pm (lower case).
tm		A date & time conversion—months in 2 digits, with leading zeroes as necessary.
tD		A date & time conversion—date as %tm%td%ty
	08	Eight characters in width, with leading zeroes as necessary.
	+	Includes sign, whether positive or negative.
	,	Includes locale-specific grouping characters.
	-	Left-justified..
	.3	Three places after decimal point.
	10.3	Ten characters in width, right justified, with three places after decimal point.

Math

La classe `Math` mette a disposizione strumenti avanzati per eseguire svariate operazioni matematiche. Al suo interno si trovano:

- Costanti fondamentali come `Math.E` e `Math.PI`
- Metodi per operazioni di base, tra cui:
 - Calcolo del valore assoluto tramite `abs`
 - Arrotondamento tramite `round`
 - Determinazione del valore minimo o massimo tra due numeri con `min` e `max`
 - `double ceil(double d)` per ottenere l'intero più piccolo che sia maggiore o uguale al parametro
 - `double floor(double d)` per calcolare l'intero più grande che sia minore o uguale
 - `double rint(double d)` per trovare l'intero più vicino a `d`

Sono presenti metodi per calcoli esponenziali e logaritmici come:

- `double exp(double d)` per calcolare e^d
- `double log(double d)` per il logaritmo naturale $\ln(d)$
- `double pow(double base, double exponent)` per eseguire l'elevamento a potenza di una base per un esponente
- `double sqrt(double d)` per calcolare la radice quadrata

Per le funzioni trigonometriche di base e inverse troviamo altri metodi come:

- `sin`, `cos`, `tan`, `asin`, `acos`, `atan`
- `double atan2(double y, double x)` che permette di convertire coordinate rettangolari in coordinate polari, restituendo l'angolo theta calcolato
- `double toDegrees(double d)`, e `toRadians(double d)` converte l'argomento in gradi o radianti

Come argomento per ogni metodo viene passato un `double` che rappresenta l'angolo espresso in radianti

Per generare numeri casuali nell'intervallo 0 - 1 si usa `Math.random()`, il cui risultato può essere scalato moltiplicandolo per un intero. Nel caso si voglia generare una serie più articolata di numeri casuali è consigliato l'utilizzo della classe dedicata `java.util.Random`

Stringhe

La classe `String` rappresenta sequenze testuali ed è caratterizzata dalla sua immutabilità, ossia il valore dell'istanza non può in alcun modo essere alterato una volta portato a compimento il suo processo di creazione.

Ogni letterale scritto tra virgolette nel codice Java è a tutti gli effetti una vera e propria istanza predefinita della classe `String`

Una stringa può essere pensata e gestita logicamente come un array di caratteri; infatti, per accedere agli elementi si usa il metodo `charAt()`, che restituisce il carattere posizionato all'indice specificato

☰ Esempio di `charAt()`

```
char[] helloArray = { 'h', 'e', 'l', 'l', 'o', '.' };
String helloString = new String(helloArray);
System.out.println(helloString);
```

Il metodo `charAt(int i)` di `String` restituisce il carattere alla *i*-esima posizione

Metodi della classe `String`

La classe presenta diversi metodi:

- `length()` restituisce la lunghezza della stringa in caratteri
- `contains(CharSequence s)` verifica la presenza di una particolare sequenza testuale con risultato booleano
- `indexOf(String s)` restituisce l'indice dal quale inizia la sottostringa `s`, -1 se non esiste la sottostringa

- `indexOf(String s, int i)` restituisce l'indice dal quale inizia la sottostringa `s` a partire dall'*i*-esimo carattere, -1 se non esiste la sottostringa
- `replace(CharSequence s1, CharSequence s2)` consente di scambiare sequenze di caratteri esatti (`s1` è quella da cercare, `s2` quella con cui sostituirla)
- `matches(String regex)` restituisce `true` se la stringa corrisponde all'espressione regolare `regex`
- `startsWith` ed `endsWith` verificano in modo rapido le sequenze collocate agli estremi di una stringa, vengono usati in congiunzione ad altri metodi
- `equals(Object o)` è l'operatore di uguaglianza fatto per le stringhe, si deve assolutamente evitare l'uso dell'operatore di uguaglianza classico tra oggetti (`==`)
- `int compareTo(String str)`, `int compareToIgnoreCase(String str)` permettono un confronto funzionale all'ordinamento, il secondo si usa quando non è necessario il case-sensitive
- `substring(int b, int e)`, `substring(int b)` permette di estrarre e generare una porzione della stringa a partire da un indice iniziale fino alla fine, o limitandosi a un indice finale escluso
- `trim` rimuove gli spazi bianchi inutili presenti all'inizio e alla fine
- `toLowerCase()`, `toUpperCase()` convertono globalmente il formato delle lettere rispettivamente in minuscolo e maiuscolo

≡ Esempio di stringa palindroma

```
public class StringDemo {
    public static void main(String[] args) {
        String palindrome = "Dot saw I was Tod";
        int len = palindrome.length();
        char[] tempCharArray = new char[len];
        char[] charArray = new char[len];
        // La stringa originale viene messa in un array di char
        for (int i = 0; i < len; i++) {
            tempCharArray[i] = palindrome.charAt(i);
        }
        // L'array di char viene girato
        for (int j = 0; j < len; j++) {
            charArray[j] = tempCharArray[len - 1 - j];
        }
        //Viene messa in un nuovo oggetto e stampata
        String reversePalindrome = new String(charArray);
        System.out.println(reversePalindrome);
    }
}
```

```
}
}
```

Conversione da numeri a stringhe

La conversione da un formato stringa verso effettivi valori numerici è operabile utilizzando i metodi di parsing specifici implementati per ogni tipo primitivo:

```
int i = Integer.parseInt("42");
float f = Float.parseFloat("3.14");
double d = Double.parseDouble("4.32144");
Float fo = Float.valueOf("3.14");
```

Se l'esigenza è quella di trasformare un numero nativo in una stringa, si farà ricorso ai metodi di conversione:

```
int i=3; double d=3.4;
String s3 = Integer.toString(i);
String s4 = Double.toString(d);

int i=3;
String s1 = String.valueOf(i);
```

Classe StringBuilder

Ogni qual volta sia necessario costruire o manipolare stringhe di cui si vuole modificare costantemente il contenuto, è mandatorio ricorrere alla classe `StringBuilder`

La classe lavora con una lunghezza variabile che consente l'aggiunta o la modifica continua di nuovi caratteri

Metodi e costruttori StringBuilder

Metodi di StringBuilder

`length()` restituisce la lunghezza della stringa presente nell'oggetto `StringBuilder`
`capacity()` restituisce la capacità attuale di questo oggetto `StringBuilder` (`>=length()`)

Costruttori di StringBuilder

Costruttore	Descrizione
<code>StringBuilder()</code>	Crea uno <code>StringBuilder</code> vuoto
<code>StringBuilder(CharSequence cs)</code>	Crea uno <code>StringBuilder</code> a partire da una sequenza di caratteri
<code>StringBuilder(int initCapacity)</code>	Crea uno <code>StringBuilder</code> con una capacità iniziale <i>initCapacity</i>
<code>StringBuilder(String s)</code>	Crea uno <code>StringBuilder</code> a partire da una stringa <i>s</i>

- `StringBuilder append(...)` aggiunge alla fine dello `StringBuilder` l'argomento (viene convertito in `String` anche i tipi primitivi)
- `StringBuilder delete(int start, int end)`, `StringBuilder deleteCharAt(int index)` cancella una porzione o un carattere della stringa
- `StringBuilder insert(int offset, ...)` : inserisce l'argomento a partire dalla posizione `offset`
- `StringBuilder replace(int start, int end, String s)`, `void setCharAt(int index, char c)` sostituisce una porzione o un singolo carattere della stringa
- `StringBuilder reverse()` inverte l'ordine dei caratteri della stringa
- `String toString()` : restituisce una stringa che contiene la sequenza dei caratteri in `StringBuilder`

≡ Esempio con `Stringbuilder`, stringa palindroma

```
public class StringBuilderDemo {
    public static void main(String[] args) {
        String palindrome = "Dot saw I was Tod";
        StringBuilder sb = new StringBuilder(palindrome);
        sb.reverse(); // reverse it
        System.out.println(sb);
    }
}
```

Espressioni regolari

Un'**espressione regolare** rappresenta a livello logico una parola capace di denotare un linguaggio regolare, utile per verificare se una data stringa sia o meno conforme alle regole di quel linguaggio.

📘 Diverse espressioni regolari nel linguaggio Java

Espressioni regolari (caratteri)

x	The character <i>x</i>
\\	The backslash character
\0n	The character with octal value <i>0n</i> ($0 \leq n \leq 7$)
\0nn	The character with octal value <i>0nn</i> ($0 \leq n \leq 7$)
\0mnn	The character with octal value <i>0mnn</i> ($0 \leq m \leq 3, 0 \leq n \leq 7$)
\xhh	The character with hexadecimal value <i>0xhh</i>
\uhhhh	The character with hexadecimal value <i>0xhhhh</i>
\t	The tab character (' <code>\u0009</code> ')
\n	The newline (line feed) character (' <code>\u000A</code> ')
\r	The carriage-return character (' <code>\u000D</code> ')
\f	The form-feed character (' <code>\u000C</code> ')
\a	The alert (bell) character (' <code>\u0007</code> ')
\e	The escape character (' <code>\u001B</code> ')
\cx	The control character corresponding to <i>x</i>

28

Espressioni regolari (chars classes)

- [abc]** a, b, or c (simple class)
- [^abc]** Any character except a, b, or c (negation)
- [a-zA-Z]** a through z or A through Z, inclusive (range)
- [a-d[m-p]]** a through d, or m through p: [a-dm-p] (union)
- [a-z&&[def]]** d, e, or f (intersection)
- [a-z&&[^bc]]** a through z, except for b and c: [a-dz] (subtraction)
- [a-z&&[^m-p]]** a through z, and not m through p: [a-lq-z](subtraction)

29

Espressioni regolari (default classes)

- . Any character (may or may not match line terminators)

\d A digit: [0-9]

\D A non-digit: [^0-9]

\s A whitespace character: [\t\n\x0B\f\r]

\S A non-whitespace character: [^\s]

\w A word character: [a-zA-Z_0-9]

\W A non-word character: [^\w]

30

Espressioni regolari (boundary)

^ The beginning of a line

\$ The end of a line

\b A word boundary

\B A non-word boundary

\A The beginning of the input

\G The end of the previous match

\Z The end of the input but for the final terminator, if any

\z The end of the input

31

Espressioni regolari (greedy operator)

$X?$ X , once or not at all

X^* X , zero or more times

X^+ X , one or more times

$X\{n\}$ X , exactly n times

$X\{n,\}$ X , at least n times

$X\{n,m\}$ X , at least n but not more than m times

32

Espressioni regolari (quotation)

$\backslash$ Nothing, but quotes the following character

$\backslash Q$ Nothing, but quotes all characters until $\backslash E$

$\backslash E$ Nothing, but ends quoting started by $\backslash Q$

33

Espressioni regolari (logical operators)

`XY` `X` followed by `Y`

`X|Y` Either `X` or `Y`

`(X)` `X`, as a capturing group

34

Dalla classe `String`, due metodi sono utilizzabili con le varie espressioni regolari:

- `split(String regex)` restituisce un array di `String` dividendo dove c'è il match con `regex`
- `replaceAll(String regex, String r)` sostituisce tutte le sequenze che corrispondono all'espressione regolare `regex` con `r`

Classe `Pattern`

Per utilizzi che richiedono procedure di validazione più strutturate o intensive, si ricorre esplicitamente alla classe `Pattern`, la quale costituisce una rappresentazione compilata in memoria di un'espressione regolare specificata tramite stringa

Partendo da un pattern già compilato, è possibile generare un oggetto derivato di tipo `Matcher`, il cui compito primario è quello di eseguire attivamente il matching, cercando in tutti i modi di far combaciare la sequenza di caratteri passata in input con le direttive fornite dall'espressione regolare pre-compilata

≡ Esempio di invocazione

```
Pattern p = Pattern.compile("a*b");
Matcher m = p.matcher("aaaaab");
```

```
boolean b = m.matches();
```

Classe Matcher

La classe `Matcher` esegue le operazioni di matching su una sequenza di caratteri in funzione dell'espressione regolare compilata

Un matcher viene creato da un pattern invocando il metodo `matcher` del pattern. Una volta creato, un matcher può essere utilizzato per eseguire tre diversi tipi di operazioni di match:

- `match` viene usato per tentare l'abbinamento esatto dell'intera sequenza in input
- `lookingAt` cerca un abbinamento parziale avendo però l'obbligo di partire rigorosamente dall'inizio
- `find` serve a scorrere e analizzare progressivamente l'input alla ricerca della prima sottosequenza utile che corrisponda positivamente

Ognuno di questi metodi restituisce un valore booleano che indica l'esito positivo o negativo.

Un matcher trova corrispondenze in un sottoinsieme del suo input chiamato regione. Per impostazione predefinita, la regione contiene tutto l'input del matcher. Una regione può essere modificata tramite il metodo `region` e costantemente monitorata interrogando i valori forniti da `regionStart` e `regionEnd`

Lo stato esplicito di un matcher include gli indici di inizio e fine dell'ultimo match (`start` ed `end`), includendo anche gli indici di inizio e fine della sotto-sequenza dell'ultimo match, nonché un conteggio totale (`groupCount`) di tali sotto-sequenze.

Per comodità, vengono forniti anche metodi per restituire queste sottosequenze in forma di stringa (`group`)

≡ Esempi per la classe matcher

```
//Esempio N.1
Matcher matcher1 = pattern.matcher("dlkfllsASDaslsdSD"); //deve
corrispondere l'intera stringa
System.out.println(matcher1.matches());
Matcher matcher2 = pattern.matcher("dlkfllsASDaslsdSD 8798767"); //la
corrispondenza deve partire dall'inizio ma non è necessario che
corrisponda l'intera stringa
System.out.println(matcher2.lookingAt());
Matcher matcher3 = pattern.matcher("dlkfllsASDaslsdSD");
//cicla su tutti i matching
while (matcher3.find()) {
```

```

        System.out.println(matcher3.group() + ": " +
            matcher3.start() + "-" + matcher3.end());
    }
    //Esempio N.2
    //Una sequenza di numeri seguita da 1 o massimo 3 lettere min.
    String regexp = "([0-9]+)([a-z]{1,3})";
    Pattern pattern2 = Pattern.compile(regexp);
    String str = "9843989jf 39203920jie 32122i";
    Matcher matcher4 = pattern2.matcher(str);
    while (matcher4.find()) {
        //restituisce il numero di gruppi (individuati dalle parentesi
        //nella regexp)
        int gc = matcher4.groupCount();
        //il gruppo 0 corrisponde all'intero matching
        for (int i = 0; i <= gc; i++) {
            System.out.println(matcher4.group(i) + ": " +
                matcher4.start(i) + "-" + matcher4.end(i));
        }
        System.out.println();
    }
}

```

Collection

Una **collection** è un oggetto volto a racchiudere più oggetti al suo interno per poterli memorizzare, recuperare ed elaborare.

In parole povere, rappresenta un gruppo di cose che vanno tenute insieme, come:

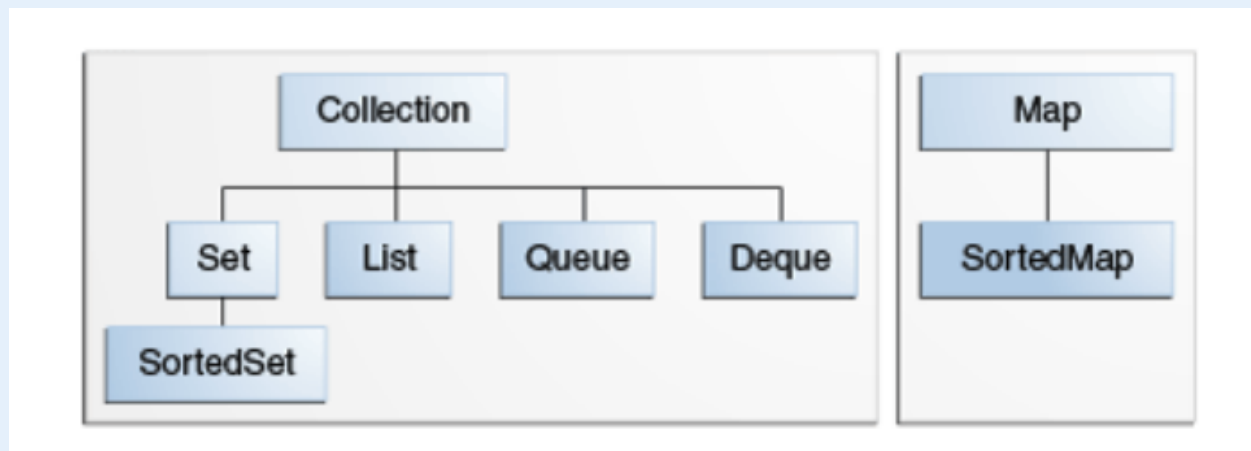
- Un mazzo di carte;
- L'elenco della rubrica telefonica;
- Insieme di email ricevute;
- etc...

Java mette a disposizione un **framework** (insieme di strumenti e librerie predefiniti che forniscono una struttura di base per lo sviluppo e la distribuzione di applicazioni in un particolare ambiente) per la gestione delle Collection, composto da:

- Interfacce
- Implementazioni
- Algoritmi (ricerca, ordinamento etc.) in grado di funzionare in modo polimorfo

Interfacce di collection

Gerarchia delle interfacce di Collection



Map è una collection particolare che non eredita da Collection

Ognuna di queste interfacce ha un diverso utilizzo:

- `Collection` è la radice della gerarchia e per questo la più generica, rappresenta semplicemente un contenitore di oggetti (elementi) senza alcun particolare vincolo
- `Set` rappresenta un contenitore di tipo insieme, non può contenere duplicati
 - `SortedSet` è un `Set` in cui gli elementi sono ordinati in ordine crescente
- `List` è una lista di elementi, ogni elemento avrà una posizione nella lista, ammette duplicati
- `Queue` è una coda in cui gli elementi hanno un preciso ordine di inserimento e recupero (si usa la tecnica FIFO, ma ci sono code particolari dette con priorità il cui ordine è dettato da una funzione di ordinamento)
- `Deque` è simile ad una coda ma permette l'accesso ad entrambe l'estremità della coda
- `Map` permette di collegare dei valori a delle chiavi (queste non possono essere duplicate all'interno della stessa `Map`)
 - `SortedMap` è una `Map` in cui le chiavi sono ordinate in ordine crescente

Metodi delle Collection generali

Tutte le interfacce che ereditano da `Collection` ereditano i suoi metodi primitivi per gestire un gruppo di oggetti:

- `int size()` restituisce il numero di oggetti
- `boolean isEmpty()` restituisce true se la `Collection` è vuota
- `boolean contains(Object element)` restituisce true se la collection contiene elementi
- `boolean add(E element)` aggiunge un elemento alla `Collection`
- `boolean remove(Object element)` rimuove un elemento alla `Collection`

- `Iterator iterator()` restituisce un oggetto `Iterator` che permette di iterare su tutti gli elementi della `Collection`

Alcuni metodi agiscono sull'intera `Collection`:

- `boolean containsAll(Collection c)` restituisce `true` se la `collection` contiene tutti gli elementi in `c`
- `boolean addAll(Collection c)` aggiunge tutti gli elementi in `c` alla `collection`
- `boolean removeAll(Collection c)` rimuove tutti gli elementi di `c` dalla `collection`
- `boolean retainAll(Collection c)` mantiene nella `collection` solo gli elementi presenti in `c`
- `void clear()` elimina tutti gli elementi dalla `collection`

L'interfaccia `Collection` ha due metodi `toArray` che permettono di fare da ponte tra le `collection` e gli array:

- `Object[] a = c.toArray()` trasforma la `collection c` in un array di oggetti, `a` avrà la stessa dimensione di `c`
- `String[] a = c.toArray(new String[0])`: se conosciamo il tipo di elementi in `c` possiamo creare un array che contiene gli stessi elementi di `c` e dello stesso tipo

Iterazione e interfaccia `Iterator`

Esistono due modi per iterare una `Collection`:

1. Il metodo `for-each`
2. Il metodo `iterator`, che utilizza l'interfaccia `Iterator` con i suoi diversi metodi tra cui
 - `hasNext()` restituisce `true` se ci sono altri elementi da visionare
 - `next()`: restituisce il prossimo elemento
 - `remove()`: rimuove l'elemento corrente

Classe `Iterator`

```
public interface Iterator {
    boolean hasNext();
    E next();
    void remove();
}
```

Filtrare elementi da una `collection`

```

Iterator it=collection.iterator()
while (it.hasNext()) {
    if (!cond(it.next())) it.remove(); //cond è un metodo fittizio
    che decide se un elemento rispetta o meno una particolare condizione
}

```

Nel caso quindi sia necessario rimuovere degli elementi è preferibile utilizzare un `Iterator` e non il metodo `for-each`

Set

Come detto precedentemente, la sottoclasse `Set` rappresenta un contenitore di tipo insieme che non può contenere duplicati. Implementa tutti i metodi dell'interfaccia `Collection` e presenta tre implementazioni:

- **HashSet**: un set implementato da una tabella hash non mantiene l'ordine di inserimento degli elementi; è l'implementazione più efficiente
- **TreeSet**: un set implementato con una struttura ad albero che mantiene l'ordine di inserimento, è meno efficiente
- **LinkedHashSet**: un set implementato con una tabella hash e puntatore che mantiene l'ordine di inserimento degli elementi

L'uguaglianza degli oggetti in questa sottoclasse è definita dai metodi `equals` e `hashCode` (restituisce un intero) della classe `Object`

Siccome `Set` non può contenere duplicati, torna particolarmente utile nella creazione di una `Collection` senza duplicati partendo da una esistente

≡ Esempio di Collection senza duplicati

```
Set s=new LinkedHashSet(c);
```

`s` non conterrà duplicati

≡ Esempio di utilizzo di Set

```

public class Esempio1 {
    public static void main(String[] args) {

```

```
// Crea un nuovo Set (insieme) di stringhe utilizzando
LinkedHashSet.

Set<String> set = new LinkedHashSet<>();

// Aggiunge elementi al set.
set.add("a");
set.add("a"); // I duplicati vengono ignorati in un Set.
Questa operazione non ha effetto.
set.add("b");
set.add("c");

// Output: "3: [a, b, c]".
// La dimensione è 3 perché il secondo "a" non è stato
inserito.
System.out.println(set.size() + ": " + set);

// Rimuove l'elemento "a" dal set.
set.remove("a");

// Output: "2: [b, c]".
// Il set ora contiene solo 2 elementi.
System.out.println(set.size() + ": " + set);

// Crea un secondo Set chiamato set1 e ci aggiunge "b" e "c".
Set<String> set1 = new LinkedHashSet<>();
set1.add("b");
set1.add("c");

// removeAll elimina dal primo 'set' tutti gli elementi
presenti nel 'set1'.
// Dato che 'set' conteneva [b, c] e 'set1' contiene [b, c],
il primo set si svuota.
set.removeAll(set1);

// Output: "0: []".
// La dimensione è 0 e l'insieme è vuoto.
System.out.println(set.size() + ": " + set);
```



```
}
}
```

Quando si crea un Set, è possibile assegnare una variabile Set senza specificare quale si usi, per rendere il codice più generico possibile e non legarlo all'implementazione. Questo vale anche per tutte le implementazioni successive di `Collection`

Operazione sugli insiemi

Il set permette operazioni su insiemi, tra cui unione, intersezione e differenza

Unione

```
Set<Type> union = new HashSet<Type>(s1);
union.addAll(s2);
```

Il metodo `addAll(Collection c)` aggiunge tutti gli elementi della collezione specificata (in questo caso `s2`) all'insieme su cui viene chiamato, se non sono già presenti

Intersezione

```
Set<Type> intersection = new HashSet<Type>(s1);
intersection.retainAll(s2);
```

Il metodo `retainAll(Collection c)` mantiene nell'insieme corrente *solo* gli elementi che sono contenuti anche nella collezione specificata (`s2`). Rimuove tutto il resto.

Differenza

```
Set<Type> difference = new HashSet<Type>(s1);
difference.removeAll(s2);
```

Il metodo `removeAll(Collection c)` rimuove dall'insieme corrente tutti gli elementi che sono contenuti anche nella collezione specificata (`s2`).

Lista

Una lista è una sequenza di elementi ordinata, in questo caso sono ammessi duplicati (purchè si trovino in posizioni diverse) e sono presenti dei metodi oltre quelli previsti da Collection per sfruttare il suo ordinamento

Una lista non ha una dimensione predefinita, cresce dinamicamente

Implementazioni delle liste

Ci sono due implementazioni:

- `ArrayList` : lista basata sugli array, la più performante
- `LinkedList` : implementazioni con doppi puntatori

Metodi lista

Alcuni metodi sfruttano l'accesso alla posizione:

- `get(int i)` : restituisce l'oggetto alla posizione `i` (le posizioni partono da 0)
- `set(int i, E element)` : inserisce l'elemento alla posizione `i`
- `remove(int i)` : elimina l'oggetto in posizione `i`
- `add(int i, E element)` : aggiunge l'elemento in posizione `i`
- `addAll(int i, Collection c)` : aggiunge tutti gli elementi in `c` a partire dalla posizione `i`

Per ricercare degli elementi all'interno di una lista si usano questi metodi:

- `indexOf(Object o)` : restituisce la posizione in cui si trova l'oggetto `o` , altrimenti -1.
Usa il metodo `equals` sotto.
- `lastIndexOf(Object o)` : restituisce l'ultima posizione in cui si trova l'oggetto `o` , altrimenti -1

`o` potrebbe trovarsi in più posizioni nella lista, se questo è duplicato viene restituita l'ultima posizione

Iterazione nella lista

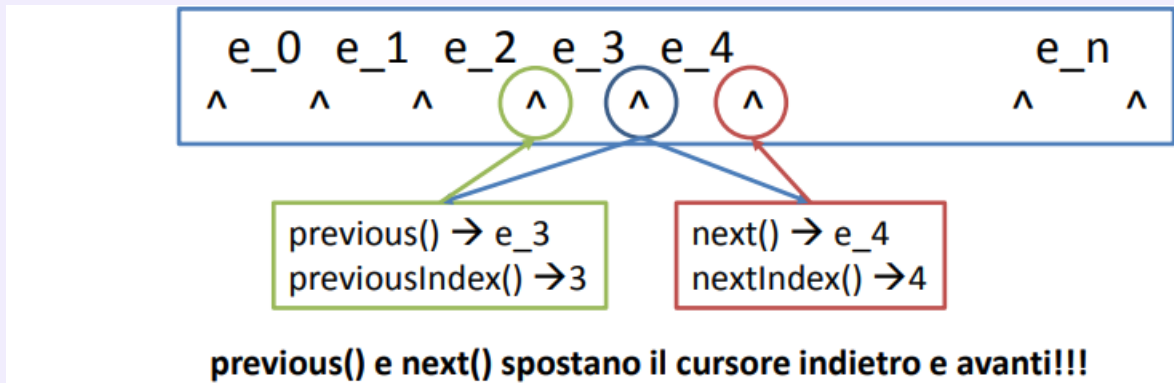
Per iterare gli elementi della lista esistono diversi metodi:

- `iterator()` : restituisce un iteratore su questa lista, gli elementi verranno elencati in sequenza
- Ci sono due metodi che restituiscono un `ListIterator` che permette di scorrere la lista sia in avanti che indietro:
 - `listIterator()` : restituisce un `ListIterator` che parte dall'inizio della lista
 - `listIterator(int i)` : restituisce un `ListIterator` che parte dalla posizione `i`

ListIterator

Il `ListIterator` è un iteratore che ammette un cursore tra un elemento e l'altro della lista (se non specificato si mette all'inizio ovviamente) e permette di andare all'elemento precedente `previous()` e all'elemento successivo `next()`

≡ ListIterator in forma grafica



`ListIterator` ha altri metodi implementati:

- `hasNext()` : booleano che controlla se è presente un elemento avanti
- `hasPrevious()` : booleano che controlla se è presente un elemento indietro
- `remove()` : rimuove l'elemento ottenuto dall'ultima chiamata di `next()` o `previous()`
- `add(E e)` : aggiunge un elemento tra `previous()` e `next()`
- `set(E e)` : sostituisce con `e` l'elemento ottenuto dall'ultima chiamata di `next()` o `previous()`

≡ Esempio di operazioni base su una lista

```
public class List1 {
    public static void main(String[] args) {
        // Inizializza una nuova ArrayList di tipo String
        List<String> list = new ArrayList<>();

        // Aggiunge elementi in coda alla lista
        list.add("a");
        list.add("b");
        list.add("c");
        list.add("a");
        System.out.println(list); // Stampa: [a, b, c, a]

        // Sostituisce l'elemento all'indice 0 ("a") con "z"
        list.set(0, "z");
        System.out.println(list); // Stampa: [z, b, c, a]
    }
}
```

```

        // Inserisce "d" esattamente all'indice 3, spostando in avanti
        gli elementi successivi
        list.add(3, "d");
        System.out.println(list); // Stampa: [z, b, c, d, a]

        // Recupera e stampa l'elemento all'indice 1 ("b")
        System.out.println(list.get(1));

        // Rimuove l'elemento all'indice 2 (che attualmente è "c")
        list.remove(2);
        System.out.println(list); // Stampa: [z, b, d, a]

        // Cerca e stampa l'indice della prima occorrenza di "a"
        System.out.println(list.indexOf("a")); // Stampa: 3

        // Aggiunge un'altra "a" in fondo e cerca l'indice della sua
        ultima occorrenza
        list.add("a");
        System.out.println(list.lastIndexOf("a")); // Stampa: 4
    }
}

```

☰ Esempio di ListIterator

```

public class List2 {
    public static void main(String[] args) {
        // Crea e popola la lista
        List<String> list = new ArrayList<>();
        list.add("a");
        list.add("b");
        list.add("c");
        list.add("a");
        list.add("z");
        list.add("h");

        // Crea un ListIterator associato alla lista
        ListIterator<String> lit = list.listIterator();
    }
}

```

```

        // hasNext() verifica se c'è un elemento successivo nella
        lista
        while (lit.hasNext()) {
            // Stampa l'indice precedente (-1 alla prima iterazione) e
            l'indice successivo
            System.out.print(lit.previousIndex() + "\t" +
            lit.nextIndex() + " -> ");

            // next() recupera l'elemento corrente e sposta il cursore
            in avanti
            System.out.println(lit.next());
        }
    }
}

```

☰ Esempio di scorrimento all'indietro

```

public class List3 {
    public static void main(String[] args) {
        // Crea e popola la lista
        List<String> list = new ArrayList<>();
        list.add("a");
        list.add("b");
        list.add("c");
        list.add("a");
        list.add("z");
        list.add("h");

        // Inizializza l'iteratore partendo dal PENULTIMO elemento
        (size - 1)
        ListIterator<String> lit = list.listIterator(list.size() - 1);

        // hasPrevious() verifica se ci sono elementi precedenti
        rispetto al cursore
        while (lit.hasPrevious()) {
            // previous() recupera l'elemento e sposta il cursore
            all'indietro

```

```

        System.out.println(list.previous());
    }
}

```

Sotto lista

Il metodo `subList(int start, int end)` permette di ottenere una sottolista a partire dalla posizione `start` fino alla posizione `end` (non inclusa).

`subList()` restituisce una nuova lista con gli elementi richiesti, non viene fatta nessuna modifica sulla lista di partenza

Algoritmi sulle liste

La classe `Collections` (da non confondere con quella classica `Collection`) mette a disposizione dei metodi statici per effettuare degli algoritmi sulle classi che implementano `Collection`:

- `sort`: ordina gli elementi nella lista
- `shuffle`: permuta in modo casuale gli elementi nella lista
- `reverse`: inverte l'ordine degli elementi
- `rotate`: ruota gli elementi di una lista
- `swap`: scambia due elementi nella lista
- `replaceAll`: sostituisce un elemento con un altro valore
- `fill`: sostituisce tutti gli elementi con un altro valore
- `copy`: copia tra due liste
- `binarySearch`: ricerca un elemento nella lista utilizzando la ricerca binaria
- `indexOfSubList`, `lastIndexOfSubList`: cercano una sottolista all'interno di una lista

☰ Esempio di shuffling

```

public class List4 {
    public static void main(String[] args) {
        // Inizializzazione della lista con un nome più chiaro
        List<String> list = new ArrayList<>();

        // Aggiunta degli elementi
        list.add("pippo");
        list.add("topolino");
        list.add("paperino");
    }
}

```

```

// Stampa iniziale: [pippo, topolino, paperino]
System.out.println(list);

// Ordina la lista in ordine alfabetico
Collections.sort(list);
System.out.println(list); // [paperino, pippo, topolino]

// Mescola gli elementi in modo casuale
Collections.shuffle(list);
System.out.println(list);

// Ruota gli elementi. Con -1, il primo elemento diventa
l'ultimo
Collections.rotate(list, -1);
System.out.println(list);
    }
}

```

SortedSet

Visione veloce di SortedSet

```

public interface SortedSet<E> extends Set<E>
{
    // Range-view
    SortedSet<E> subSet(E fromElement, E
toElement);
    SortedSet<E> headSet(E toElement);
    SortedSet<E> tailSet(E fromElement);

    // Endpoints
    E first();
    E last();

    // Comparator access
    Comparator<? super E> comparator();
}

```

Gli elementi sono ordinati in modo decrescente in base al loro ordinamento naturale o può essere passato un Comparator nel costruttore

Le range-view permettono viste sugli oggetti sfruttando il loro ordinamento

Il primo e l'ultimo elemento del set

Permette di accedere al Comparator di questo set

☰ Ricerca frequenza parole con SortedSet

```
import java.util.HashMap;
import java.util.Map;

public class Freq {
    public static void main(String[] args) {
        // Inizializza una nuova HashMap.
        // La Chiave (String) sarà la parola, il Valore (Integer) sarà
        // il suo conteggio.
        Map<String, Integer> m = new HashMap<>();

        // Inizializza la tabella delle frequenze leggendo dalla riga
        // di comando (args)
        for (String a : args) {
            // Cerca la parola 'a' nella mappa.
            // Se la parola non è ancora stata inserita, m.get()
            // restituirà null.
            Integer freq = m.get(a);

            // L'operatore ternario (condizione ? caso_vero :
            // caso_falso) decide cosa inserire:
            // - Se freq è null (parola nuova), inserisce la parola
            // con valore 1.
            // - Se freq NON è null (parola già vista), reinserisce la
            // parola
            // sovrascrivendo il vecchio valore con freq + 1.
            m.put(a, (freq == null) ? 1 : freq + 1);
        }

        // m.size() restituisce il numero di chiavi (ovvero le parole
        // uniche)
        System.out.println(m.size() + " distinct words:");

        // Stampa il contenuto dell'intera mappa nel formato
        // {chiave1=valore1, chiave2=valore2...}
        System.out.println(m);
    }
}
```



```
}
}
```

Queue

L'interfaccia `Queue`, come già detto precedentemente, crea una coda per poter mantenere degli elementi e processarli in un preciso ordine (di solito FIFO).

Essendo un'interfaccia, non è possibile istanziare un oggetto di tipo `Queue`, ma estende una `Collection` già presente.

Generalmente questa interfaccia implementa dei metodi che creano delle eccezioni o fanno il return di un valore specifico (null o false)

Metodi in Queue

- `add(E e)` : aggiunge un elemento alla coda, restituisce `true` se l'elemento è inserito altrimenti genera un'eccezione
- `element()` : restituisce l'elemento in testa alla coda senza rimuoverlo, se la coda è vuota genera un'eccezione
- `offer(E e)` : aggiunge un elemento alla coda, in caso contrario offre valore null
- `peek()` restituisce l'elemento in testa alla coda senza rimuoverlo, oppure null se la coda è vuota
- `poll()` : restituisce e rimuove l'elemento in testa alla coda, null se la coda è vuota
- `remove()` : restituisce e rimuove l'elemento in testa alla coda, se la coda è vuota genera un'eccezione

I metodi di inserimento, cancellazione e recupero sono duplicati e si differenziano sul loro comportamento (eccezione/restituzione valore) nel caso in cui la coda è vuota o ha raggiunto la capacità massima

☰ Esempio operazioni base

```
public class Queue1 {
    public static void main(String[] args) {
        // Crea una nuova coda FIFO (First-In-First-Out) basata su
        LinkedList
        Queue<String> q = new LinkedList<>();

        // offer() aggiunge un elemento in fondo alla coda.
        q.offer("g");
        q.offer("h");
        q.offer("a");
    }
}
```

```

        // peek() recupera il primo elemento della coda.
        // Poiché "g" è stato inserito per primo, sarà il primo della
        fila.
        System.out.println(q.peek()); // Stampa: g
    }
}

```

Implementazioni di Queue

L'implementazioni di default di una coda è quella di `LinkedList`, che implementa sempre la coda FIFO. Nel caso ci sia bisogno di una lista con priorità, si usa `PriorityQueue`, l'elemento sempre in cima è quello più piccolo.

☰ Esempio coda con priorità

```

public class Queue2 {
    public static void main(String[] args) {
        // Inizializza una PriorityQueue. Gli elementi verranno
        ordinati
        // automaticamente in base alla loro "priorità" (ordine
        alfabetico).
        Queue<String> q = new PriorityQueue<>();

        // Aggiunge elementi alla coda
        q.offer("g");
        q.offer("h");
        q.offer("a");

        // peek() guarda il primo elemento in coda, .
        // Poiché "a" viene prima di "g" e "h" nell'alfabeto, è lui
        // l'elemento con la priorità più alta.
        System.out.println(q.peek()); // Stampa: a
    }
}

```

Deque (Deck)

Deque è una Collection che è simile alla coda, ma con l'eccezione di poter inserire elementi all'inizio e alla fine di essa.

Metodi di Deque

Rispetto alla coda, Deque implementa le sue controparti:

- `addFirst(E e)`, `addLast(E e)` : funziona come la controparte di `Queue`, una aggiunge in cima alla coda e il secondo alla fine delle coda (fa risultare un eccezione nel caso non sia possibile)
- `offerFirst(E e)`, `offerLast(E e)` funziona come la controparte di `Queue`, una aggiunge in cima alla coda e il secondo alla fine delle coda (valore null nel caso non sia possibile)
- `removeFirst()`, `removeLast()` funziona come la controparte di `Queue`, una rimuove in cima alla coda e il secondo alla fine delle coda (fa risultare un eccezione nel caso non sia possibile)
- `pollFirst()`, `pollLast()` funziona come la controparte di `Queue`, una rimuove in cima alla coda e il secondo alla fine delle coda (valore null nel caso non sia possibile)
- `getFirst()`, `getLast()` funziona come la controparte di `Queue`, una prende il valore in cima alla coda e il secondo alla fine della coda (fa risultare un eccezione nel caso non sia possibile)
- `peekFirst()`, `peekLast()` funziona come la controparte di `Queue`, una prende il valore in cima alla coda e il secondo alla fine della coda (valore null nel caso non sia possibile)

≡ Esempio di Deque

```
public class Deque1 {
    public static void main(String[] args) {
        // Inizializza una Deque utilizzando l'implementazione
        LinkedList.
        Deque<String> q = new LinkedList<>();

        // offerLast() aggiunge elementi in coda (alla fine della
        lista)
        q.offerLast("g"); // Stato attuale: [g]
        q.offerLast("h"); // Stato attuale: [g, h]

        // offerFirst() aggiunge un elemento in testa (all'inizio
        della lista),
        // spingendo indietro gli elementi già presenti.
        q.offerFirst("z"); // Stato attuale: [z, g, h]
```

```

// Aggiunge un altro elemento in coda
q.offerLast("a"); // Stato attuale: [z, g, h, a]

// peekLast() legge l'ultimo elemento (senza rimuoverlo)
System.out.println("Coda: " + q.peekLast()); // Stampa:
a

// peekFirst() legge il primo elemento (senza rimuoverlo)
System.out.println("Testa: " + q.peekFirst()); // Stampa:
z
    }
}

```

Map

L'interfaccia MAP permette di creare delle associazioni chiave-valore. Ogni chiave è univoca e ad ognuna di essa è associato un solo valore.

Esistono tre implementazioni dell'interfaccia Map (simili a Set):

- HashMap
- TreeMap
- LinkedHashMap

Metodi di Map

L'interfaccia Map implementa alcuni metodi utili

- `get(Object key)` : restituisce l'oggetto associato alla chiave `key`
- `put(K key, V value)` : inserisce una coppia chiave-valore nella `Map`
- `remove(Object key)` : rimuove la coppia chiave-valore associata a `key`
- `containsKey(Object key)` , : booleano che restituisce `true` se la `Map` contiene la chiave `key`
- `containsValue(Object value)` : booleano che restituisce `true` se la `Map` contiene il valore `value`
- `putAll(Map<K, V> map)` : inserisce tutti gli elementi di `map`

☰ **Calcolare le frequenze con una HashMap**

```

public class Freq {
    public static void main(String[] args) {
        // Inizializza una nuova HashMap.
        // La Chiave (String) sarà la parola, il Valore (Integer) sarà
        // il suo conteggio.
        Map<String, Integer> m = new HashMap<>();

        // Inizializza la tabella delle frequenze leggendo dalla riga
        // di comando (args)
        for (String a : args) {
            // Cerca la parola 'a' nella mappa.
            // Se la parola non è ancora stata inserita, m.get()
            // restituirà null.
            Integer freq = m.get(a);

            // L'operatore ternario (condizione ? caso_vero :
            // caso_falso) decide cosa inserire:
            // - Se freq è null (parola nuova), inserisce la parola
            // con valore 1.
            // - Se freq NON è null (parola già vista), reinserisce la
            // parola
            // sovrascrivendo il vecchio valore con freq + 1.
            m.put(a, (freq == null) ? 1 : freq + 1);
        }

        // m.size() restituisce il numero di chiavi (ovvero le parole
        // uniche)
        System.out.println(m.size() + " distinct words:");

        // Stampa il contenuto dell'intera mappa nel formato
        // {chiave1=valore1, chiave2=valore2...}
        System.out.println(m);
    }
}

```

SortedMap

```
public interface SortedMap<K, V>
    extends Map<K, V>{
    Comparator<? super K>
    comparator();
```

Gli elementi sono ordinati in modo decrescente in base all'ordinamento naturale delle chiavi o può essere passato un `Comparator` nel costruttore

```
    SortedMap<K, V> subMap(K
    fromKey, K toKey);
    SortedMap<K, V> headMap(K
    toKey);
    SortedMap<K, V> tailMap(K
    fromKey);
```

Le range-view permettono viste sugli oggetti sfruttando l'ordinamento sulle chiavi

La prima e l'ultima chiave

```
    K firstKey();
    K lastKey();
}
```

Equals e hashCode

Ogni classe eredita da `Object` due importanti metodi:

- `equals()` , che ricordiamo essere il metodo per confrontare due operatori, restituisce un valore booleano `true` quando due oggetti sono uguali
Il metodo `equals` è utilizzato da `Set` per evitare duplicati, ma anche da tutti i metodi di `Collection` che cercano un oggetto
- `hashCode()` , che restituisce il codice intero hash per l'oggetto. È utilizzato dalle tabelle hash come quelle fornite da `HashMap`

Se due oggetti sono uguali in base al metodo `equals (Object)` , la chiamata del metodo `hashCode` su ciascuno dei due oggetti deve produrre gli stessi risultati interi, poichè `Object` genera l'hash code utilizzando l'indirizzo di memoria dell'oggetto

Generics

Le `Collection` sono originariamente strutture generiche nate per memorizzare oggetti di tipo generico `Object` , tuttavia è possibile tipizzare queste collezioni utilizzando i generics attraverso la sintassi `<T>` , la quale permette a tutti i metodi della struttura di lavorare specificamente su oggetti di tipo `T` anziché sul tipo generico `Object`

Per comprendere l'utilità dei generics, è fondamentale analizzare come cambia la gestione dei tipi di dato nel codice:

☰ Senza vs Con Generics

Senza generics

```
List list = new ArrayList();
list.add("hello");
String s = (String)list.get(0);
```

E' necessaria l'operazione di cast (cambio del tipo di oggetto)

Con generics

```
List<String> list = new ArrayList<String>();
list.add("hello");
String s = list.get(0); // no cast
```

Il metodo get è stato tipizzato in base al generics <String>

Eccezioni

Le eccezioni sono eventi che si verificano durante l'esecuzione del programma e identifica un'anomalia che impedisce il normale flusso del programma.

In Java esiste un meccanismo (gestore delle eccezioni) che cattura e gestisce le eccezioni, che di solito possono essere generate durante l'esecuzione di un metodo. Il gestore delle eccezioni gestisce soltanto un determinato tipo di eccezione.

Ogni eccezione ha un oggetto con tutte le informazioni su cosa è accaduto.

Tipologie di eccezioni

Esistono 3 grandi tipologie di eccezioni:

- **Exception (con sottoclassi)**: sono eccezioni che possono essere gestite e catturate (anche se il programmatore dovrebbe anticipare e gestire questo tipo di eccezioni)
- **Error (e sottoclassi)**: sono eccezioni che non possono essere gestite e dipendono da qualcosa esterno dal programma
- **RuntimeException (e sottoclassi)**: eccezioni interne al programma che non possono essere anticipate o catturate (essenzialmente sono dei bug)

Tutte le eccezioni sono sottoclassi di Exception, ed è possibile creare una propria classe exception, estendendo quella originale o una delle sue sottoclassi

Try-catch and finally

Per catturare un'eccezione bisogna utilizzare il blocco try-catch

Blocco try-catch

```
try {
    //istruzioni
} catch (Exception ex) {
    //gestione dell'eccezione
} finally {
    //istruzioni che devono essere indipendentemente eseguite dal
    successo o non successo della try-catch
}
```

Il blocco `finally` è opzionale, può essere utilizzato per effettuare delle operazioni di “pulizia”

Per gestire più eccezioni nella stessa linea di codice si usa la seguente sintassi:

Catch di più eccezioni

```
try {
    //Istruzioni
} catch (IOException|SQLException ex) {
    System.err.println("Caught Exception: " + ex.getMessage());
}
```

`System.err.println()` stampa l'errore ma non ferma l'esecuzione del programma.

Lancio di un eccezione

I metodi possono dichiarare il tipo di eccezione che si potrebbe verificare al suo interno. Questo in sostanza obbliga tutti i metodi chiamanti a gestire l'eccezione nel blocco try-catch

```
public File openFile(String filename) throws IOException{
    try {
        //istruzioni
    } catch (IOException ex) {
        throw ex;
    }
}
```


Il throw generalmente si usa per imporre regole rigide (come dei parametri importanti non validi, file non trovati, connessioni perse) in modo da interrompere il flusso di esecuzione.

Una guida interessante con esempi ancora più chiari la trovate [qui](#)

I/O Stream

Un flusso I/O rappresenta una sorgente di input o una destinazione di output e sono di vario tipo, possono essere file su disco, array di memoria ecc...

Gli **stream** supportano molti tipi diversi di dati, dai byte semplici agli oggetti. Alcuni flussi semplicemente si occupano di **trasmettere dati**, mentre altri **manipolano** i dati.

Ma indipendentemente da come funzionano, tutti i flussi presentano **una sequenza di dati**.

Byte stream

I byte stream sono utilizzati per leggere e scrivere byte (8 bit) su un dispositivo di I/O. Ereditano dalle classi **InputStream** e **OutputStream**.

Ci sono diverse classi che ereditano direttamente dai byte stream per scrivere e leggere sui file:

- FileInputStream
- FileOutputStream

Ad esempio:

```
public class CopyBytes{
    public static void main(String[] args) throws IOException{
        FileInputStream in = null;
        FileOutputStream out = null;
        try{
            in = new FileInputStream("sorgente.txt");
            out = new FileOutputStream("destinazione.txt");
            int c;
            while((c = in.read()) != -1)
                out.write(c);
        }
        finally{
            if(in != null)
                in.close();
            if(out != null)
                out.close();
        }
    }
}
```

```

    }
}

```

Come possiamo notare si **chiudono sempre** gli stream. E' di vitale importanza eseguire ciò, poiché possiamo deallocare risorse e evitiamo di perdere le informazioni scritte sullo stream.

Lo stream sui byte andrebbe evitato poiché **non è un'ottima operazione**. Essa è un operazione di basso livello ed esistono degli stream **idonei** per scrivere e leggere caratteri per caratteri.

Character stream

Si utilizza in caso dovessimo eseguire operazioni IO di caratteri. Questo tipo di flusso gestisce automaticamente la codifica corretta per i caratteri, seguendo lo stile UTF o ISO.

Le classi di questo tipo di stream sono:

- FileReader
- FileWriter

Ad esempio:

```

FileReader inputStream = null;
FileWriter outputStream = null;

try{
    inputStream = new FileReader("sorgente.txt");
    outputStream = new FileWriter("destinazione.txt");

    int c;
    while ((c = inputStream.read())!=-1){
        outputStream.write(c);
    }
} finally{
    if(inputStream != null){
        inputStream.close();
    }
    if(outputStream != null){
        outputStream.close();
    }
}
}

```

Line-oriented

Per facilitare il lavoro in un file pieno di caratteri (e generalmente i file presentano più testi che caratteri), andiamo a raccogliere proprio i testi.

Per far ciò dobbiamo accedere a sequenze di caratteri, ovvero dobbiamo **prelevare le stringhe**.

Le classi IO che permettono il prelievo di una stringa e l'inserimento di una stringa su un file sono:

- `BufferedReader`
- `PrintWriter`

Ad esempio:

```
BufferedReader inputStream = null;
PrintWriter outputStream = null;

try{
    inputStream = new BufferedReader(new FileReader("sorgente.txt"));
    outputStream = new PrintWriter(new FileWriter("destinazione.txt"));

    String l;
    while ((l = inputStream.readLine()) != null){
        outputStream.println(l);
    }
} finally{
    if(inputStream != null){
        inputStream.close();
    }
    if(outputStream != null){
        outputStream.close();
    }
}
```

Buffered I/O

Il più famoso ed utilizzato, specialmente per lo sviluppo in JAVA, è proprio il **buffered** stream.

E' molto conveniente poiché le operazioni vengono eseguite **direttamente sul dispositivo di I/O**; questo è possibile poiché le classi di JAVA (generalmente) sono **unbuffered**.

- **Unbuffered Stream:** Ogni singola richiesta di lettura o scrittura viene gestita e inviata **direttamente e immediatamente al dispositivo di I/O** (come il disco rigido)
- **Buffered Stream:** Utilizza un'area di memoria temporanea e velocissima nella RAM (chiamata buffer)

Il confronto tra questi due metodi serve a dimostrare il principio dell'**ottimizzazione delle prestazioni**.

Il punto chiave è che il "collo di bottiglia" in informatica è quasi sempre la comunicazione con l'hardware fisico. Accedere al disco fisso o inviare un pacchetto di rete costa tantissimo in termini di tempo macchina.

Le classi buffered, ovvero `BufferedReader` e `BufferedWriter`) utilizzano un buffer predisposto all'I/O che velocizza le operazioni di read&write.

Le funzioni che seguono questa classe sono:

- **BufferedInputStream** e **BufferedOutputStream:** creano le versioni buffered di un byte stream
- **BufferedReader** e **BufferedWriter:** creano le versioni buffered di un character stream

A livello di codice si creano nel seguente modo:

☰ Esempio di creazione `BufferedReader` e `BufferedWriter`

```
InputStream = new BufferedReader(new FileReader("pippo.txt"));
OutputStream = new BufferedWriter(new FileWriter("pluto.txt"));
```

La classe File

La classe `File` può rappresentare due concetti diversi sotto lo stesso nome:

- **nome** di un particolare file
- **nome** di una directory

Se stiamo parlando dell'ultimo caso, possiamo conoscere gli insieme dei file che lo compongono tramite il metodo **list()**, che restituisce un array di stringe con gli elementi di tale insieme.

Ovviamente è possibile anche selezionare solo un tipo di oggetti nella cartella (per esempio se vogliamo visualizzare solo tutti i `.exe`) ricorrendo ad un **filtro**, detto **directory filter**.

L'interfaccia **FilenameFilter** è molto semplice: `public interface FilenameFilter{ boolean accept (File dir, String name);}`

Le classi che implementano questa funzione devono fornire sia un metodo **accept()** obbligatoriamente sia un metodo **list()** della classe madre **File**, così da eseguire una **call back** per determinare quali nomi di file devono essere inclusi nella lista.

Metodo Accept()

Gli argomenti del metodo **accept()** sono due:

- Un oggetto **File** che rappresenta la directory in cui si trova il file
- Un oggetto **String** che rappresenta il nome del file

Precedentemente, abbiamo visto come il metodo **accept()** ci assicurava di lavorare solo con il nome del file, senza avere informazioni sul suo **percorso**.

Tuttavia, esiste un'altra interfaccia simile detta **FileFilter** in cui il metodo **accept**, **prende in input direttamente l'oggetto File**.

La classe File è usata anche per:

- creare nuove cartelle o percorsi tramite `\mkdir()` o `\mkdirs()`
- accedere alle caratteristiche dei file
- eliminare un file
- verificare l'oggetto File

I/O con l'utente

Fino ad ora questi output che abbiamo rilasciato nel programma dal file per l'utente non erano formattati adeguatamente. Per rispondere a questa necessità utilizziamo il **format**. Stessa pratica diviene necessaria per quanto riguarda l'input, in maniera tale da ottenere le singole informazioni necessarie e non tutto il file.

Per processare gli input si utilizza la classe **scanner**; essa suddivide l'input in **token** e traduce ogni token in un tipo predefinito.

La suddivisione tra i vari token avviene tramite l'utilizzo dei **white space**.

```
Scanner s = null;
double sum = 0;
try{
    s = new Scanner(new BufferedReader(new FileReader("input.txt")));
    while (s.hasNext()){
        if( s.hasNextDouble()){ //ricerchiamo il dato semplice Double
            sum+=s.nextDouble();
        } else s.next();
    }
} finally{
    s.close();
}
```

```
}
System.out.println(sum);
```

La scomposizione della "Matrioska":

1. Il Minatore: `new FileReader("input.txt")`

- **Il suo unico lavoro:** Creare il collegamento fisico con il file sul disco rigido e iniziare a estrarre i dati.
- **Il problema:** Questo è uno stream **unbuffered** (come abbiamo visto precedentemente). Estrae i caratteri uno per volta, direttamente dal disco. Se lo usassimo da solo, sarebbe lentissimo. Non sa cosa siano le parole o i numeri, vede solo una fila di caratteri.

2. Il Trasportatore: `new BufferedReader(...)`

- **Il suo unico lavoro:** Ottimizzare la velocità. Prende in input il lento `FileReader`, aspetta che estragga un bel po' di caratteri, li accumula in un "magazzino" nella RAM (il buffer) e poi li passa al livello successivo tutti in blocco.
- **Il problema:** È velocissimo, ma è "stupido". Sa leggere intere frasi, ma per lui è solo testo puro. Se nel file c'è scritto "100", per il `BufferedReader` è la parola composta dalle lettere '1', '0', '0', non il numero matematico cento.

3. L'Interprete: `new Scanner(...)`

- **Il suo unico lavoro:** Tradurre e analizzare. Prende il flusso di testo veloce e ottimizzato che gli arriva dal `BufferedReader` e lo "scansiona".
- **Il vantaggio:** È lui che ti mette a disposizione i metodi comodi come `.nextInt()`, `.nextDouble()` o `.nextLine()`. Riesce a capire gli spazi, ad andare a capo e a convertire il testo nei tipi di dato Java che ti servono.

Se Java avesse un blocco monolitico come `FileScanner`, dovresti riscrivere tutto il codice usando un ipotetico `WebScanner`.

Invece, grazie a questo sistema a matrioska, **cambi solo il pezzo più interno** (il minatore), lasciando intatto il trasportatore e l'interprete. Si sostituisce il `FileReader` con uno stream di rete, ma lo `Scanner` continuerà a funzionare esattamente allo stesso modo nel resto del tuo programma.

I/O da linea di comando

La classe `System` mette a disposizione tre stream collegati al terminale:

- `System.in`: `InputStream` che legge l'input
- `System.out`: `PrintStream` che stampa l'output
- `System.err`: `PrintStream` che stampa messaggi di errore

```

public static void main(String args[]){
    Scanner scanner=new Scanner(new InputStreamReader(System.in));
    String s = "";
    while (scanner.hasNext()){
        s= scanner.next();
        if(!s.equalsIgnoreCase("exit")){
            System.out.println("Hai scritto: "+s);
        } else{
            System.out.println("Goodbye!");
            break;
        }
    }
    scanner.close();
}

```

Data Streams

I flussi di dati supportano l'I/O binario dei valori di dati primitivi, tra cui le stringhe pure.

I flussi di dati implementano l'interfaccia **DataInput** o **DataOutput**, tra cui le più utilizzate di queste due interfacce sono proprio **DataInputStream** e **DataOutputStream**.

L'esempio **DataStreams** mostra i flussi di dati scrivendo un **set di record di dati** e quindi rilegendoli. Ogni record è costituito da tre valori relativi ad un articolo:

- Prezzo (double),
- quantità (int),
- descrizione (String).

Ad esempio:

```

static final String dataFile="invoicedata";

static final double[] prices={19.99,9.99,15.99,4.99};
static final int[] units={12,8,13,29,50};
static final String[] descs={
    "Java T-shirt",
    "Java Mug",
    "Duke Juggling Dolls",
    "Java Pin",
};

```

`DataStream` rileva una condizione di fine file catturando una **`EOFException`**, anziché testare un valore restituito non valido, come i modelli visti precedentemente. Generalmente si usa **`IOException (end of stream)`**.

Ogni scrittura in `DataStream` corrisponde esattamente alla lettura corrispondente passo passo. Il programmatore deve assicurarsi che i tipi di output ed input siano abbinati correttamente. I dati in input sono binari, senza nulla che indichi il tipo dei singoli valori o da dove inizia il flusso.

Serializzazione degli oggetti

Se volessimo invece salvare dei dati su un file che non siano di tipo primitivo, come gli oggetti, `DataStream` da solo non basta. Dovremmo memorizzare tutte le parti di un oggetto in maniera separata, con una rappresentazione ben precisa così da ricostruire l'informazione correttamente al momento del bisogno. Questo processo risulta noioso e faticoso e prende il nome di **persistenza**.

Definizione persistenza

La **persistenza** di un oggetto è la capacità dell'oggetto di vivere separatamente dal programma che lo ha generato

Java contiene un meccanismo interno per creare oggetti persistenti, detto **serializzazione**. La serializzazione trasforma in una sequenza di byte il concetto che vogliamo rappresentare, dopo di che questa rappresentazione di byte può essere ricostruita nel suo aspetto originale. Dopo che l'oggetto viene serializzato può essere salvato su un file o inviato ad un altro PC.

La sua realizzazione prevede l'uso di un interfaccia con due classi:

- **`ObjectOutputStream`**, il quale contiene il metodo **`writeObject`** che serve per serializzare un oggetto.
- **`ObjectInputStream`**, il quale contiene il metodo **`readObject`** che serve per deserializzare un oggetto.

Ogni oggetto che si vuole serializzare deve implementare l'interfaccia **`Serializable`**, la quale non contiene metodi e **serve soltanto al compilatore** per comprendere che un oggetto di quella determinata classe può essere serializzato.

`ObjectInputStream` e **`ObjectOutputStream`** sono **stream di manipolazione** e devono essere utilizzati congiuntamente a un **`OutputStream`** e un **`InputStream`**.

Ne sussegue l'esempio:

```
FileOutputStream outFile = new FileOutputStream("info.dat");
ObjectOutputStream outStream = new ObjectOutputStream (outFile);
```



```
outStream.writeObject(myCar); // dove myCar è un oggetto di una classe Car
definita dal programmatore
```

Per poter leggere l'oggetto serializzato e ricaricarlo in memoria centrale si procederà come segue:

```
FileInputStream inFile = new FileInputStream("info.dat");
ObjectInputStream inStream = new ObjectInputStream(inFile);
Car myCar = (Car) inStream.readObject();
```

La serializzazione di un oggetto si occupa di serializzare tutti gli eventuali riferimenti ad esso collegati. Dunque, se la classe Car contenesse dei riferimenti (variabili di classe o di istanza) a oggetti di classe Engine, questa verrebbe serializzata automaticamente e diverrebbe parte della serializzazione di Car. La classe Engine dovrà, pertanto, implementare anch'essa l'interfaccia serializable al suo interno.

⚡ Errore comune:

Gli attributi di classe, cioè definiti come static, **non vengono serializzati**. Per poterli salvare occorre provvedere in modo personalizzato.

☰ Esempio di serializzazione

```
class Nave implements Serializable {

    private static int nroNavi = 1;
    private int nroNave;
    private String nomeNave;

    // Costruttore: assegna il numero progressivo e il nome alla nave
    Nave(String nomeNave) {
        nroNave = nroNavi++; // Assegna il numero corrente e incrementa
        // il contatore
        this.nomeNave = nomeNave; // Assegna il nome passato come
        // parametro
    }

    // Restituisce una rappresentazione testuale della nave
    public String toString() {
        return nomeNave + ": " + nroNave;
    }
}
```

```

    }

    // Metodo per SALVARE (serializzare) l'oggetto su file binario
    "info.dat"

    public void salva() throws FileNotFoundException, IOException {
        ObjectOutputStream out = new ObjectOutputStream(new
        FileOutputStream("info.dat"));
        out.writeObject(this); // Serializza l'oggetto Nave corrente
        out.writeObject(nroNavi);
        out.close();
    }

    // Metodo statico per CARICARE (deserializzare) un oggetto Nave
    dal file "info.dat"

    public static Nave carica() throws FileNotFoundException,
    IOException {
        ObjectInputStream in = new ObjectInputStream(new
        FileInputStream("info.dat"));
        Nave n = (Nave) in.readObject(); // Legge e ricostruisce
        l'oggetto Nave
        Nave.nroNavi = in.readObject();
        in.close();
        return n; // Restituisce la nave deserializzata
    }
}

```

Molte classi della **libreria standard Java implementano l'interfaccia Serializable** in modo da essere serializzate quando necessario, come ad esempio la classe String.

Transient

A volte, quando si serializza un oggetto, si può desiderare di **escludere delle informazioni**, ad esempio, una password.

Questo accade quando le informazioni vengono trasmesse via rete, poiché il pericolo è che, pur dichiarandola con visibilità privata, la password possa essere letta e usata da soggetti non autorizzati quando viene serializzata.

Per questo tipo di richiesta e problematica ci viene in aiuto la parola chiave **transient** associato al nome di una variabile. Esso indica al compilatore di nascondere quella variabile, così da non rappresentarla come parte dello stream di byte della versione serializzata dell'oggetto.

Ad esempio:

```
private transient String password;  
private String CF;
```

In questo caso, durante la serializzazione dell'oggetto che include queste due variabili, la variabile CF, anche se privata, sarà serializzata, mentre la password sarà ignorata nella rappresentazione grazie alla keyword transient.